

Data Warehouse Macrofinanciero
Proyecto Guiado A→Z

Francisco Ramírez

16 de junio de 2026

Índice general

Capítulo 1

Hito 1: Creación del Proyecto

1.1. Objetivo

El objetivo de este hito es construir la estructura inicial del proyecto y preparar el entorno de desarrollo que utilizaremos durante todo el manual.

Al finalizar este hito el estudiante tendrá:

- Un repositorio Git inicializado.
- Una estructura de carpetas profesional.
- Un entorno virtual de Python.
- Dependencias básicas instaladas.
- El primer commit del proyecto.

1.2. Resultado esperado

Al finalizar este hito, la estructura del proyecto deberá lucir de la siguiente forma:

```
dwh_macrofinanciero_rd/
```

```
+-- data/  
|   +-- raw/  
|   \-- processed/  
  
+-- sql/
```

```
+++ etl/  
+++ app/  
+++ docs/  
+++ tests/  
  
+++ README.md  
+++ requirements.txt  
+++ .gitignore
```

1.3. Paso 1: Crear la carpeta del proyecto

Abrir PowerShell y navegar al directorio donde se almacenará el proyecto.
Por ejemplo:

```
cd C:\Users\t490\Documents\MacroPol
```

Crear la carpeta principal:

```
mkdir dwh_macrofinanciero_rd  
cd dwh_macrofinanciero_rd
```

Verificar la ubicación actual:

```
pwd
```

1.4. Paso 2: Abrir el proyecto en VS Code

Desde la terminal:

```
code .
```

VS Code deberá abrir la carpeta recién creada.

1.5. Paso 3: Inicializar Git

Abrir la terminal integrada de VS Code y ejecutar:

```
git init
```

Salida esperada:

```
Initialized empty Git repository
```

Verificar el estado:

```
git status
```

1.6. Paso 4: Crear la estructura de carpetas

Crear las carpetas principales:

```
mkdir data
mkdir sql
mkdir etl
mkdir app
mkdir docs
mkdir tests
```

Crear las subcarpetas para almacenamiento de datos:

```
mkdir data\raw
mkdir data\processed
```

1.7. Paso 5: Crear los archivos iniciales

Crear los siguientes archivos:

- README.md
- requirements.txt
- .gitignore
- .env.example

La estructura del proyecto deberá verse así:

```
dwh_macrofinanciero_rd/
|
+-- data/
+-- sql/
+-- etl/
+-- app/
+-- docs/
+-- tests/
|
+-- README.md
+-- requirements.txt
+-- .gitignore
+-- .env.example
```

1.8. Paso 6: Configurar el archivo .gitignore

Crear el archivo:

```
.gitignore
```

con el siguiente contenido:

```
.venv/  
.env  
  
**pycache**/  
  
*.pyc  
*.pyo  
  
.vscode/  
  
data/processed/  
  
.DS_Store  
Thumbs.db
```

1.9. Paso 7: Crear el README inicial

Archivo:

```
README.md
```

Contenido sugerido:

```
# Data Warehouse Macrofinanciero RD  
  
Proyecto practico para construir una plataforma  
de inteligencia macrofinanciera utilizando:  
  
* PostgreSQL  
* Python  
* Streamlit  
* GitHub
```

Autor: Francisco Ramirez

1.10. Paso 8: Crear el entorno virtual

Crear el entorno virtual:

```
python -m venv .venv
```

Activarlo:

Windows:

```
.venv\Scripts\activate
```

Linux/Mac:

```
source .venv/bin/activate
```

La terminal deberá mostrar:

```
(.venv)
```

1.11. Paso 9: Instalar dependencias mínimas

Instalar los paquetes necesarios:

```
pip install pandas  
pip install sqlalchemy  
pip install psycopg2-binary  
pip install python-dotenv
```

Guardar las dependencias instaladas:

```
pip freeze > requirements.txt
```

1.12. Paso 10: Realizar el primer commit

Agregar todos los archivos al repositorio:

```
git add .
```

Verificar:

```
git status
```

Crear el primer commit:

```
git commit -m "Initial_project_structure"
```

Consultar el historial:

```
git log --oneline
```

Salida esperada:

```
3f2a9ab Initial project structure
```

1.13. Checklist de validación

Antes de continuar al siguiente hito, verificar:

- ☐ VS Code instalado y funcionando.
- ☐ Proyecto creado.
- ☐ Git inicializado.
- ☐ Estructura de carpetas creada.
- ☐ Entorno virtual activo.
- ☐ Dependencias instaladas.
- ☐ Primer commit realizado.

1.14. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Un proyecto abierto en VS Code.
- Un repositorio Git funcional.
- Un entorno virtual configurado.
- Una estructura base lista para desarrollar el Data Warehouse.

1.15. Próximo hito

En el Hito 2 construiremos la infraestructura de base de datos:

- Instalación de PostgreSQL.
- Creación de la base de datos.
- Configuración de variables de entorno.
- Primera conexión Python → PostgreSQL.

Capítulo 2

Hito 2: Instalación y Configuración de PostgreSQL

2.1. Objetivo

El objetivo de este hito es preparar el motor de base de datos que servirá como núcleo del Data Warehouse macrofinanciero.

Al finalizar este hito el estudiante será capaz de:

- Instalar PostgreSQL.
- Crear una base de datos para el proyecto.
- Conectarse desde VS Code.
- Configurar variables de entorno.
- Conectarse desde Python.
- Ejecutar consultas SQL básicas.

2.2. Resultado esperado

Al finalizar este hito deberá existir:

- Una instancia funcional de PostgreSQL.
- Una base de datos llamada `macrofinanzas`.
- Un usuario con permisos adecuados.

CAPÍTULO 2. HITO 2: INSTALACIÓN Y CONFIGURACIÓN DE POSTGRESQL10

- Un archivo `.env` para gestionar credenciales.
- Un script Python capaz de conectarse a la base de datos.

2.3. Paso 1: Verificar instalación de PostgreSQL

Abrir PowerShell y ejecutar:

```
psql --version
```

Salida esperada:

```
psql (PostgreSQL) 17.x
```

Si el comando no funciona:

- Verificar que PostgreSQL está instalado.
- Verificar que el directorio `bin` está en el `PATH`.

2.4. Paso 2: Iniciar PostgreSQL

Verificar que el servicio está activo.

En Windows:

1. Abrir Servicios.
2. Buscar PostgreSQL.
3. Verificar estado Running".

Alternativamente:

```
net start postgresql-x64-17
```

2.5. Paso 3: Conectarse al servidor

Abrir terminal:

```
psql -U postgres
```

Introducir la contraseña configurada durante la instalación.

Debería aparecer:

```
postgres=#
```

2.6. Paso 4: Crear la base de datos

Ejecutar:

```
CREATE DATABASE macrofinanzas ;
```

Verificar:

```
\l
```

La nueva base deberá aparecer en la lista.

Salir:

```
\q
```

2.7. Paso 5: Conectarse a la nueva base

```
psql -U postgres -d macrofinanzas
```

Verificar:

```
SELECT current_database();
```

Resultado esperado:

```
macrofinanzas
```

2.8. Paso 6: Crear archivo .env

En la raíz del proyecto crear:

```
.env
```

Contenido:

```
DB_HOST=localhost  
DB_PORT=5432  
DB_NAME=macrofinanzas  
DB_USER=postgres  
DB_PASSWORD=tu_password
```

Importante:

El archivo `.env` nunca debe enviarse a GitHub.

2.9. Paso 7: Actualizar .env.example

Crear una versión pública:

`.env.example`

Contenido:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=macrofinanzas
DB_USER=postgres
DB_PASSWORD=your_password
```

Este archivo sí puede subirse al repositorio.

2.10. Paso 8: Crear módulo de conexión

Crear:

`etl/db.py`

```
from sqlalchemy import create_engine
from dotenv import load_dotenv
import os

load_dotenv()

HOST = os.getenv("DB_HOST")
PORT = os.getenv("DB_PORT")
DB = os.getenv("DB_NAME")
USER = os.getenv("DB_USER")
PASSWORD = os.getenv("DB_PASSWORD")

connection_string = (
    f"postgresql+psycopg2://{USER}:{PASSWORD}@{HOST}:{PORT}/{DB}"
)

engine = create_engine(connection_string)

print("Conexion_creada_correctamente.")
```

2.11. Paso 9: Probar la conexión

Ejecutar:

```
python etl/db.py
```

Salida esperada:

Conexión creada correctamente.

2.12. Paso 10: Realizar una consulta desde Python

Modificar temporalmente:

```
from sqlalchemy import text

with engine.connect() as conn:
    resultado = conn.execute(
        text("SELECT version();")
    )

    '''
    for fila in resultado:
        print(fila)
    '''
```

Ejecutar nuevamente:

```
python etl/db.py
```

Resultado esperado:

PostgreSQL 17.x ...

2.13. Paso 11: Instalar extensiones de VS Code

Instalar:

- PostgreSQL
- SQLTools
- SQLTools PostgreSQL Driver

Estas extensiones facilitarán la ejecución de consultas y exploración de objetos.

2.14. Paso 12: Crear primer script SQL

Crear:

sql/test_connection.sql

Contenido:

```
SELECT current_database();
```

```
SELECT current_user;
```

```
SELECT version();
```

Ejecutarlo desde VS Code o psql.

2.15. Checklist de validación

Antes de continuar verificar:

- ☐ PostgreSQL instalado.
- ☐ Servicio iniciado.
- ☐ Base de datos creada.
- ☐ Archivo .env configurado.
- ☐ Python conectado a PostgreSQL.
- ☐ Consulta SQL ejecutada correctamente.

2.16. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Base de datos **macrofinanzas**.
- Módulo de conexión Python.
- Variables de entorno configuradas.
- VS Code conectado a PostgreSQL.
- Primer script SQL ejecutado exitosamente.

2.17. Próximo hito

En el Hito 3 construiremos la primera capa de datos:

- Diseño de las fuentes.
- Creación de datos simulados.
- Organización de carpetas **raw** y **processed**.
- Primera ingesta hacia PostgreSQL.

Capítulo 3

Hito 2: Configuración de DuckDB como Base de Datos Analítica Local

3.1. Objetivo

El objetivo de este hito es preparar la base de datos analítica que servirá como núcleo del proyecto.

A diferencia de PostgreSQL, DuckDB no requiere instalación como servicio ni privilegios de administrador. La base de datos se almacena como un archivo local dentro del proyecto, lo cual la convierte en una excelente opción para trabajar en laptops corporativas con restricciones de instalación.

Al finalizar este hito el estudiante será capaz de:

- Instalar DuckDB desde Python.
- Crear una base de datos local en formato `.duckdb`.
- Conectarse a DuckDB desde Python.
- Ejecutar consultas SQL básicas.
- Crear un módulo reutilizable de conexión.
- Verificar que el proyecto puede operar sin PostgreSQL.

3.2. Resultado esperado

Al finalizar este hito deberá existir una base de datos local:

```
data/warehouse/macrofinanzas.duckdb
```

y un archivo de conexión:

```
etl/db.py
```

capaz de abrir la base de datos desde Python.

3.3. ¿Por qué DuckDB?

DuckDB es una base de datos analítica embebida.

Esto significa que:

- No requiere servidor.
- No requiere instalación con privilegios de administrador.
- No necesita usuarios ni contraseñas.
- Puede vivir como un archivo dentro del proyecto.
- Se integra directamente con Python, Pandas y Streamlit.
- Es muy eficiente para consultas analíticas.

En este proyecto utilizaremos DuckDB como motor local para construir el Data Warehouse macrofinanciero.

3.4. Arquitectura del hito

La nueva arquitectura será:

```
CSV
|
V
Python
|
V
```

CAPÍTULO 3. HITO 2: CONFIGURACIÓN DE DUCKDB COMO BASE DE DATOS ANALÍTICA LO

```
DuckDB
|
V
Streamlit
```

La base quedará almacenada como archivo:

```
macrofinanzas.duckdb
```

3.5. Paso 1: Actualizar la estructura del proyecto

Dentro del proyecto crear la carpeta:

```
data/warehouse
```

En PowerShell:

```
mkdir data\warehouse
```

La estructura deberá quedar así:

```
dwh_macrofinanciero_rd/

+-- data/
|   +-- raw/
|   +-- processed/
|   -- warehouse/

+-- sql/
+-- etl/
+-- app/
+-- docs/
+-- tests/
```

3.6. Paso 2: Instalar DuckDB

Con el entorno virtual activo:

```
pip install duckdb
```

También instalaremos paquetes que utilizaremos en el proyecto:

```
pip install pandas
pip install python-dotenv
```

Guardar dependencias:

```
pip freeze > requirements.txt
```

3.7. Paso 3: Verificar instalación

Ejecutar:

```
python
```

Dentro de Python:

```
import duckdb
```

```
print(duckdb.**version**)
```

Salir:

```
exit()
```

Si se imprime la versión, DuckDB está correctamente instalado.

3.8. Paso 4: Crear archivo de configuración

Aunque DuckDB no requiere usuario ni contraseña, utilizaremos un archivo `.env` para mantener centralizada la ruta de la base.

Crear:

```
.env
```

Contenido:

```
DUCKDB_PATH=data/warehouse/macrofinanzas.duckdb
```

Importante: el archivo `.env` no debe subirse a GitHub.

3.9. Paso 5: Actualizar `.env.example`

Crear o actualizar:

```
.env.example
```

CAPÍTULO 3. HITO 2: CONFIGURACIÓN DE DUCKDB COMO BASE DE DATOS ANALÍTICA LO

Contenido:

DUCKDB_PATH=data/warehouse/macrofinanzas.duckdb

Este archivo sí puede subirse al repositorio.

3.10. Paso 6: Crear módulo de conexión

Crear:

etl/db.py

Contenido:

```
import os
import duckdb

from dotenv import load_dotenv

load_dotenv()

DB_PATH = os.getenv(
    "DUCKDB_PATH",
    "data/warehouse/macrofinanzas.duckdb"
)

def get_connection():
    conn = duckdb.connect(DB_PATH)
    return conn

if __name__ == "__main__":
    conn = get_connection()

    """
    result = conn.execute(
        "SELECT_ 'DuckDB_conectado_correctamente'_AS_mensaje;"
    ).fetchdf()

    print(result)

    conn.close()
    """
```

3.11. Paso 7: Ejecutar prueba de conexión

Desde la terminal:

```
python etl/db.py
```

Salida esperada:

```
mensaje
0 DuckDB conectado correctamente
```

Al ejecutar este script, DuckDB creará automáticamente el archivo:

```
data/warehouse/macrofinanzas.duckdb
```

3.12. Paso 8: Verificar archivo de base de datos

Confirmar que existe:

```
data/warehouse/macrofinanzas.duckdb
```

En PowerShell:

```
dir data\warehouse
```

Deberá aparecer el archivo `macrofinanzas.duckdb`.

3.13. Paso 9: Crear primer script SQL

Crear:

```
sql/test_connection.sql
```

Contenido:

```
SELECT
'DuckDB' AS motor,
current_database() AS base_actual;
```

3.14. Paso 10: Ejecutar SQL desde Python

Crear:

```
etl/run_sql_file.py
```

Contenido:

```
from db import get_connection

def run_sql_file(path):
    conn = get_connection()

    """
    with open(path, "r", encoding="utf-8") as file:
        sql = file.read()

    result = conn.execute(sql).fetchdf()

    print(result)

    conn.close()
    """

if __name__ == "__main__":
    run_sql_file(
        "sql/test_connection.sql"
    )
```

Ejecutar:

```
python etl/run_sql_file.py
```

Salida esperada:

```
motor          base_actual
0 DuckDB      macrofinanzas
```

3.15. Paso 11: Instalar extensión recomendada en VS Code

Buscar e instalar en VS Code:

- DuckDB SQL Tools
- Python

Esto permitirá explorar archivos DuckDB y ejecutar consultas desde el editor.

3.16. Paso 12: Crear consulta de prueba

Crear:

`sql/check_duckdb.sql`

Contenido:

```
SELECT  
1 AS prueba ,  
'ok' AS estado ;
```

Modificar temporalmente `etl/run_sql_file.py` para ejecutar este archivo o reutilizar la función:

```
run_sql_file(  
    "sql/check_duckdb.sql"  
)
```

3.17. Diferencias importantes frente a PostgreSQL

Al usar DuckDB, algunas cosas cambian:

- No necesitamos servidor.
- No necesitamos usuarios.
- No usamos contraseñas.
- No dependemos del puerto 5432.
- La base vive en un archivo.
- El proyecto es más portable.

Sin embargo, mantendremos la lógica conceptual del Data Warehouse:

staging -> core -> mart

DuckDB soporta schemas, por lo que seguiremos trabajando con:

- staging
- core
- mart

3.18. Buenas prácticas

Durante este proyecto seguiremos estas reglas:

1. La base DuckDB se guardará en **data/warehouse**.
2. El archivo **.env** guardará la ruta local.
3. El archivo **.env.example** documentará la configuración.
4. El módulo **etl/db.py** será el único responsable de abrir conexiones.
5. Cada script cerrará la conexión al finalizar.

3.19. Checklist de validación

Antes de continuar verificar:

- ☐ Carpeta **data/warehouse** creada.
- ☐ DuckDB instalado.
- ☐ Archivo **.env** creado.
- ☐ Archivo **.env.example** actualizado.
- ☐ Módulo **etl/db.py** creado.
- ☐ Script **etl/db.py** ejecutado correctamente.
- ☐ Archivo **macrofinanzas.duckdb** generado.
- ☐ Script SQL de prueba ejecutado.

3.20. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Una base DuckDB local.
- Un módulo de conexión reutilizable.
- Una carpeta `data/warehouse`.
- Un archivo `.env.example`.
- Un mecanismo básico para ejecutar SQL desde Python.

3.21. Próximo hito

En el Hito 3 trabajaremos con las primeras fuentes de datos del proyecto. Construiremos:

- Archivos CSV iniciales.
- Lectura desde Python.
- Validaciones básicas.
- Organización de datos crudos y procesados.

Capítulo 4

Hito 3: Diseño de las Fuentes de Datos e Ingesta Inicial

4.1. Objetivo

El objetivo de este hito es construir la primera capa del flujo de datos. Al finalizar este hito el estudiante será capaz de:

- Comprender las fuentes de información del proyecto.
- Diseñar datasets alineados con las necesidades de negocio.
- Organizar correctamente las carpetas de datos.
- Crear archivos CSV simulados.
- Leer datos desde Python.
- Realizar validaciones básicas antes de cargar a PostgreSQL.

4.2. ¿Por qué comenzamos con archivos CSV?

En proyectos reales los datos suelen provenir de:

- APIs.
- Bases de datos.
- Excel.

- CSV.
- Servicios web.
- Plataformas regulatorias.

Sin embargo, durante las primeras etapas del proyecto es recomendable trabajar con archivos CSV.

Esto permite:

- Diseñar el modelo de datos.
- Construir el ETL.
- Validar la arquitectura.
- Evitar complejidad innecesaria.

Una vez validado el proceso, sustituiremos los CSV por fuentes reales.

4.3. Arquitectura de la ingesta

El flujo de trabajo será:

```
Fuente CSV
|
V
data/raw
|
V
Python
|
V
Validación
|
V
data/processed
|
V
PostgreSQL
```

4.4. Fuentes utilizadas

El proyecto utilizará dos fuentes principales.

4.4.1. Fuente 1: Tipo de Cambio

Representa información similar a la publicada por el Banco Central.

Grano:

Una observación por fecha y moneda.

Variables:

- fecha
- moneda
- tasa_compra
- tasa_venta

Ejemplo:

```
fecha,moneda,tasa_compra,tasa_venta
2024-01-01,USD,58.50,58.80
2024-01-01,EUR,63.10,63.60
```

4.4.2. Fuente 2: Balances Bancarios

Representa información similar a la publicada por la Superintendencia de Bancos.

Grano:

Una observación por mes y entidad.

Variables:

- periodo
- entidad_cod
- entidad_nombre
- tipo_entidad

- activos_totales
- cartera_creditos
- depositos
- patrimonio
- resultado_neto
- depositos_me
- cartera_me

Ejemplo:

```
periodo,entidad_cod,activos_totales
202401,B001,120000000
```

4.5. Paso 1: Crear estructura de almacenamiento

Verificar que existe:

```
data/
```

```
+-- raw/
```

```
+-- processed/
```

raw contendrá los datos originales.

processed contendrá los datos validados.

4.6. Paso 2: Crear archivo tipo_cambio.csv

Crear:

```
data/raw/tipo_cambio.csv
```

Contenido mínimo:

```
fecha,moneda,tasa_compra,tasa_venta
2024-01-01,USD,58.50,58.80
2024-01-02,USD,58.55,58.85
2024-01-03,USD,58.60,58.90
2024-01-01,EUR,63.10,63.60
2024-01-02,EUR,63.20,63.70
```

4.7. Paso 3: Crear archivo balances.csv

Crear:

data/raw/balances.csv

Contenido mínimo:

```
periodo,entidad_cod,entidad_nombre,  
tipo_entidad,activos_totales,  
cartera_creditos,depositos,  
patrimonio,resultado_netto,  
depositos_me,cartera_me
```

```
202401,B001,Banco Alfa,  
Banco Multiple,120000000,  
85000000,95000000,  
15000000,1200000,  
25000000,18000000
```

4.8. Paso 4: Crear script de exploración

Crear:

etl/explore_sources.py

```
import pandas as pd  
  
fx = pd.read_csv(  
    "data/raw/tipo_cambio.csv"  
)  
  
balances = pd.read_csv(  
    "data/raw/balances.csv"  
)  
  
print("\nTipo_de_Cambio")  
print(fx.head())  
  
print("\nBalances")  
print(balances.head())
```

4.9. Paso 5: Ejecutar exploración

Ejecutar:

```
python etl/explore_sources.py
```

Resultado esperado:

- Visualización de las primeras filas.
- Confirmación de lectura correcta.

4.10. Paso 6: Verificar estructura

Agregar:

```
print (fx.info())
```

```
print (balances.info())
```

Analizar:

- Número de filas.
- Número de columnas.
- Tipos de datos.
- Valores faltantes.

4.11. Paso 7: Validaciones básicas

Agregar:

```
print (
fx.isnull().sum()
)
```

```
print (
balances.isnull().sum()
)
```

Verificar:

- Valores nulos.

- Fechas inválidas.
- Tasas negativas.

4.12. Paso 8: Crear capa processed

Convertir tipos de datos:

```
fx["fecha"] = pd.to_datetime(
fx["fecha"]
)

fx["tasa_compra"] = (
fx["tasa_compra"]
. astype(float)
)

fx["tasa_venta"] = (
fx["tasa_venta"]
. astype(float)
)
```

4.13. Paso 9: Guardar datos procesados

```
fx.to_csv(
"data/processed/tipo_cambio.csv",
index=False
)

balances.to_csv(
"data/processed/balances.csv",
index=False
)
```

4.14. Paso 10: Documentar las fuentes

Crear:

```
docs/data_dictionary.md
```

Documentar:

- Nombre de variable.
- Descripción.
- Tipo de dato.
- Fuente.
- Frecuencia.

4.15. Entregables

Al finalizar este hito deberá existir:

- Archivos CSV en raw.
- Archivos validados en processed.
- Script de exploración.
- Diccionario de datos.
- Validaciones básicas implementadas.

4.16. Checklist de validación

- ☐ Existe carpeta raw.
- ☐ Existe carpeta processed.
- ☐ tipo_cambio.csv creado.
- ☐ balances.csv creado.
- ☐ Python puede leer ambos archivos.
- ☐ Validaciones ejecutadas.
- ☐ Archivos procesados generados.

4.17. Próximo hito

En el Hito 4 diseñaremos el modelo dimensional.
Construiremos:

- Dimensión Tiempo.
- Dimensión Moneda.
- Dimensión Entidad Financiera.
- Hecho Tipo de Cambio.
- Hecho Balance Bancario.
- Primer esquema estrella.

Capítulo 5

Hito 4: Diseño del Modelo Dimensional

5.1. Objetivo

El objetivo de este hito es diseñar el modelo dimensional del Data Warehouse macrofinanciero.

Al finalizar este hito el estudiante será capaz de:

- Identificar el grano de cada tabla de hechos.
- Diferenciar dimensiones y hechos.
- Diseñar un esquema estrella.
- Definir claves de negocio y claves subrogadas.
- Documentar el modelo antes de implementarlo en PostgreSQL.

5.2. Resultado esperado

Al finalizar este hito tendremos definido el modelo lógico del Data Warehouse.

El modelo estará compuesto por:

- `dim_tiempo`
- `dim_moneda`
- `dim_entidad_financiera`

- `fact_tipo_cambio`
- `fact_balance_mensual`

5.3. Paso 1: Entender qué es un modelo dimensional

Un modelo dimensional organiza los datos para facilitar el análisis. Está compuesto por dos tipos principales de tablas:

- **Dimensiones:** describen el contexto del análisis.
- **Hechos:** contienen medidas numéricas que queremos analizar.

Ejemplo:

- La fecha es una dimensión.
- La entidad financiera es una dimensión.
- La moneda es una dimensión.
- La tasa de cambio es una medida.
- Los activos bancarios son una medida.

5.4. Paso 2: Identificar los procesos de negocio

En este proyecto tendremos dos procesos de negocio principales:

1. Seguimiento del mercado cambiario.
2. Seguimiento de balances bancarios mensuales.

Cada proceso generará una tabla de hechos.

5.5. Paso 3: Definir el grano

El grano indica qué representa una fila de una tabla de hechos. Esta decisión es fundamental.

5.5.1. Grano de fact_tipo_cambio

Una fila por fecha y moneda.

Ejemplo:

```
2024-01-01 | USD
2024-01-01 | EUR
2024-01-02 | USD
```

5.5.2. Grano de fact_balance_mensual

Una fila por mes y entidad financiera.

Ejemplo:

```
2024-01-31 | B001
2024-01-31 | B002
2024-02-29 | B001
```

5.6. Paso 4: Definir dimensiones**5.6.1. Dimensión tiempo**

La dimensión tiempo será compartida por ambos hechos.
Esto la convierte en una dimensión conformada.

```
dim\'_tiempo
```

```
sk\_tiempo
fecha
anio
mes
anio\_mes
nombre\_mes
trimestre
es\_fin\_de_mes
```

5.6.2. Dimensión moneda

Esta dimensión describe las monedas utilizadas en el mercado cambiario.

dim_moneda

sk_moneda
moneda_cod
moneda_nombre

5.6.3. Dimensión entidad financiera

Esta dimensión describe las entidades financieras.

dim_entidad_financiera

sk_entidad
entidad_cod
entidad_nombre
tipo_entidad
vigente_desde
vigente_hasta
es_actual

La dimensión entidad financiera se diseñará para soportar cambios históricos mediante SCD Tipo 2.

5.7. Paso 5: Definir tablas de hechos

5.7.1. Hecho tipo de cambio

fact_tipo_cambio

sk_tiempo
sk_moneda
tasa_compra
tasa_venta
tasa_promedio
spread

Medidas:

- tasa_compra
- tasa_venta
- tasa_promedio
- spread

5.7.2. Hecho balance mensual

fact_balance_mensual

sk_tiempo
 sk_entidad
 activos_totales
 cartera_creditos
 depositos
 patrimonio
 resultado_neto
 depositos_me
 cartera_me

Medidas:

- activos_totales
- cartera_creditos
- depositos
- patrimonio
- resultado_netto
- depositos_me
- cartera_me

5.8. Paso 6: Definir claves

5.8.1. Clave de negocio

La clave de negocio proviene de la fuente original.
 Ejemplos:

- moneda_cod: USD, EUR.
- entidad_cod: B001, B002.
- fecha 2024-01-31.

5.8.2. Clave subrogada

La clave subrogada es creada por el Data Warehouse.
Ejemplos:

- sk_tiempo
- sk_moneda
- sk_entidad

Las tablas de hechos no deben depender directamente de las claves de negocio.

Deben conectarse mediante claves subrogadas.

5.9. Paso 7: Diseñar el esquema estrella

El esquema estrella del proyecto será:

```

dim_tiempo
|
|
dim_moneda ---- fact_tipo_cambio

      dim_tiempo
        |
        |

dim_entidad_financiera ---- fact_balance_mensual

```

La dimensión `dim_tiempo` conecta ambos hechos y permite análisis integrados.

5.10. Paso 8: Crear documento del modelo

Crear el archivo:

docs/modelo_dimensional.md

Contenido sugerido:

```
# Modelo Dimensional
```

```
## Procesos de negocio
```

1. Mercado cambiario
2. Balances bancarios mensuales

```
## Tablas de hechos
```

```
### fact_tipo_cambio
```

Grano:

Una fila por fecha y moneda.

Medidas:

- * tasa_compra
- * tasa_venta
- * tasa_promedio
- * spread

```
### fact_balance_mensual
```

Grano:

Una fila por mes y entidad financiera.

Medidas:

- * activos_totales
- * cartera_creditos
- * depositos
- * patrimonio

```

* resultado_netto
* depositos_me
* cartera_me

## Dimensiones

* dim_tiempo
* dim_moneda
* dim_entidad_financiera

```

5.11. Paso 9: Crear tabla de definición del modelo

Agregar al documento una tabla como la siguiente:

Tabla	Tipo	Grano
dim_tiempo	Dimensión	Una fila por día
dim_moneda	Dimensión	Una fila por moneda
dim_entidad_financiera	Dimensión	Una fila por versión de entidad
fact_tipo_cambio	Hecho	Una fila por fecha y moneda
fact_balance_mensual	Hecho	Una fila por mes y entidad

5.12. Paso 10: Definir indicadores derivados

A partir de este modelo construiremos indicadores como:

- Tipo de cambio promedio.
- Spread cambiario.
- ROA.
- ROE.
- Dolarización de depósitos.
- Dolarización de cartera.
- Concentración bancaria.

Fórmulas principales:

```

tasa_promedio = (tasa_compra + tasa_venta) / 2

spread = tasa_venta - tasa_compra

ROA = resultado_netto / activos_totales

ROE = resultado_netto / patrimonio

dolarizacion_depositos = depositos_me / depositos

dolarizacion_cartera = cartera_me / cartera_creditos

```

5.13. Paso 11: Validar el diseño

Antes de crear las tablas en PostgreSQL, responder:

1. ¿Cada tabla de hechos tiene un grano claro?
2. ¿Cada medida pertenece al grano definido?
3. ¿Las dimensiones describen correctamente los hechos?
4. ¿La dimensión tiempo puede usarse en ambos hechos?
5. ¿La entidad financiera requiere historia?

5.14. Checklist de validación

Antes de continuar verificar:

- ☐ Procesos de negocio identificados.
- ☐ Grano de cada hecho definido.
- ☐ Dimensiones definidas.
- ☐ Hechos definidos.
- ☐ Claves de negocio identificadas.
- ☐ Claves subrogadas definidas.
- ☐ Documento `modelo_dimensional.md` creado.

5.15. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Diseño lógico del Data Warehouse.
- Documento del modelo dimensional.
- Definición de grano.
- Lista de dimensiones.
- Lista de hechos.
- Lista de indicadores derivados.

5.16. Próximo hito

En el Hito 5 implementaremos la primera capa física del Data Warehouse:

- Creación de schemas.
- Creación de tablas staging.
- Preparación de la capa bronze.
- Primer script SQL del modelo.

Capítulo 6

Hito 5: Construcción de la Capa Staging (Bronze)

6.1. Objetivo

El objetivo de este hito es construir la primera capa física del Data Warehouse.

La capa staging será el punto de entrada de todos los datos al sistema. Al finalizar este hito el estudiante será capaz de:

- Crear schemas en PostgreSQL.
- Diseñar tablas de aterrizaje.
- Comprender la función de la capa Bronze.
- Implementar una estrategia de carga desacoplada de las transformaciones.
- Preparar el sistema para recibir datos desde Python.

6.2. Resultado esperado

Al finalizar este hito deberá existir la siguiente estructura lógica:

PostgreSQL

```
+-- staging  
|
```

```
+-- core
|
+-- mart
```

Y dentro del schema staging:

```
staging.tipo_cambio_raw

staging.balance_bancario_raw
```

6.3. ¿Qué es la capa Staging?

La capa staging es el área donde aterrizan los datos provenientes de las fuentes externas.

Su objetivo es almacenar los datos en su forma más cercana al origen.

No se realizan:

- Cálculos.
- Transformaciones complejas.
- Generación de indicadores.
- Limpieza profunda.

La regla general es:

Cargar primero. Transformar después.

6.4. Arquitectura del Data Warehouse

La arquitectura que construiremos será:

```
Fuentes
|
V
Staging (Bronze)
|
V
Core (Silver)
|
V
Mart (Gold)
```

6.5. Paso 1: Crear carpeta para scripts SQL

Verificar:

```
sql/
```

Crear:

```
sql/01_create_schemas.sql
```

6.6. Paso 2: Crear los schemas

Agregar:

```
CREATE SCHEMA IF NOT EXISTS staging;
```

```
CREATE SCHEMA IF NOT EXISTS core;
```

```
CREATE SCHEMA IF NOT EXISTS mart;
```

Guardar.

6.7. Paso 3: Ejecutar el script

Desde psql:

```
\i sql/01_create_schemas.sql
```

Verificar:

```
SELECT schema_name
FROM information_schema.schemata
WHERE schema_name IN
(
  'staging ',
  'core ',
  'mart '
);
```

Resultado esperado:

```
staging
core
mart
```


6.8. Paso 4: Crear script de tablas staging

Crear:

sql/02_staging_ddl.sql

6.9. Paso 5: Diseñar tabla tipo_cambio_raw

Agregar:

```
CREATE TABLE IF NOT EXISTS
staging.tipo_cambio_raw
(
  fecha TEXT,

  '''
moneda TEXT,

tasa_compra TEXT,

tasa_venta TEXT,

archivo_origen TEXT,

cargado_en TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
'''

);
```

6.10. ¿Por qué usamos TEXT?

Esta es una práctica común en Data Engineering.

Ventajas:

- La carga nunca falla por formato.
- Se preserva el dato original.
- La validación ocurre posteriormente.
- Se facilita el diagnóstico de errores.

6.11. Paso 6: Diseñar tabla balance_bancario_raw

Agregar:

```
CREATE TABLE IF NOT EXISTS
staging.balance_bancario_raw
(
periodo TEXT,

'''
entidad_cod TEXT,

entidad_nombre TEXT,

tipo_entidad TEXT,

activos_totales TEXT,

cartera_creditos TEXT,

depositos TEXT,

patrimonio TEXT,

resultado_neto TEXT,

depositos_me TEXT,

cartera_me TEXT,

archivo_origen TEXT,

cargado_en TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
'''

);
```

6.12. Paso 7: Ejecutar el DDL

Ejecutar:

```
\i sql/02_staging_ddl.sql
```

6.13. Paso 8: Verificar tablas creadas

Consultar:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema='staging';
```

Resultado esperado:

```
tipo_cambio_raw
```

```
balance_bancario_raw
```

6.14. Paso 9: Inspeccionar estructura

Consultar:

```
SELECT
column_name,
data_type
FROM information_schema.columns
WHERE table_schema='staging'
AND table_name='tipo_cambio_raw';
```

Verificar que todas las columnas fueron creadas correctamente.

6.15. Paso 10: Crear script de prueba

Crear:

```
sql/test_staging.sql
```

Contenido:

```
SELECT *  
FROM staging.tipo_cambio_raw;
```

```
SELECT *  
FROM staging.balance_bancario_raw;
```

Actualmente ambas tablas deben estar vacías.

6.16. Paso 11: Documentar la capa staging

Crear:

docs/staging_design.md

Documentar:

- Nombre de tabla.
- Fuente original.
- Frecuencia.
- Número esperado de columnas.
- Responsable de carga.

Ejemplo:

Tabla:
staging.tipo_cambio_raw

Fuente:
Banco Central

Frecuencia:
Diaria

Objetivo:
Almacenar datos crudos del mercado cambiario.

6.17. Paso 12: Crear consulta de monitoreo

Crear:

sql/monitor_staging.sql

Contenido:

```
SELECT
COUNT(*) AS total_filas
FROM staging.tipo_cambio_raw;
```

```
SELECT
COUNT(*) AS total_filas
FROM staging.balance_bancario_raw;
```

Esta consulta se utilizará para monitorear futuras cargas.

6.18. Buenas prácticas

Durante este proyecto seguiremos las siguientes reglas:

1. Nunca modificar datos manualmente en staging.
2. Mantener los datos lo más cercanos posible a la fuente.
3. No realizar cálculos en staging.
4. Todas las transformaciones ocurren en la capa core.
5. Conservar metadatos de carga.

6.19. Checklist de validación

Antes de continuar verificar:

- ☐ Schema staging creado.
- ☐ Schema core creado.
- ☐ Schema mart creado.
- ☐ Tabla `tipo_cambio_raw` creada.

- ☐ Tabla `balance_bancario_raw` creada.
- ☐ Scripts SQL almacenados en la carpeta `sql`.
- ☐ Consultas de validación ejecutadas.

6.20. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Arquitectura física inicial del Data Warehouse.
- Schemas creados.
- Tablas staging creadas.
- Scripts SQL documentados.
- Capa Bronze lista para recibir datos.

6.21. Próximo hito

En el Hito 6 construiremos el primer proceso ETL.
Aprenderemos a:

- Leer archivos CSV desde Python.
- Conectarnos a PostgreSQL.
- Cargar información a staging.
- Automatizar la ingesta.
- Validar que los datos fueron cargados correctamente.

Capítulo 7

Hito 5: Construcción de la Capa Staging en DuckDB

7.1. Objetivo

El objetivo de este hito es construir la primera capa física del Data Warehouse utilizando DuckDB.

La capa **staging** será el punto de entrada de todos los datos al sistema. En esta capa almacenaremos los datos provenientes de archivos CSV en una forma cercana a la fuente original.

Al finalizar este hito el estudiante será capaz de:

- Crear schemas en DuckDB.
- Crear tablas de aterrizaje para datos crudos.
- Ejecutar scripts SQL desde Python.
- Preparar la base local para recibir datos desde archivos CSV.
- Mantener la arquitectura **staging** ->**core** ->**mart**.

7.2. Resultado esperado

Al finalizar este hito deberán existir tres schemas:

```
staging
core
mart
```

y dos tablas en la capa `staging`:

```
staging.tipo_cambio_raw
```

```
staging.balance_bancario_raw
```

7.3. Paso 1: Crear script de schemas

Crear el archivo:

```
sql/01_create_schemas.sql
```

Contenido:

```
CREATE SCHEMA IF NOT EXISTS staging;
```

```
CREATE SCHEMA IF NOT EXISTS core;
```

```
CREATE SCHEMA IF NOT EXISTS mart;
```

7.4. Paso 2: Crear script para ejecutar SQL

Crear o verificar el archivo:

```
etl/run_sql_file.py
```

Contenido:

```
from db import get_connection
```

```
def run_sql_file(path):
```

```
    '''
```

```
    conn = get_connection()
```

```
    with open(path, "r", encoding="utf-8") as file:  
        sql = file.read()
```

```
    conn.execute(sql)
```

```
    conn.close()
```



```
'''  
  
if **name** == "**main**":  
  
    '''  
    run_sql_file(  
        "sql/01_create_schemas.sql"  
    )  
  
    print("Script_SQL_ejecutado_correctamente.")  
    '''
```

7.5. Paso 3: Ejecutar creación de schemas

Desde la terminal:

```
python etl/run_sql_file.py
```

Salida esperada:

```
Script SQL ejecutado correctamente.
```

7.6. Paso 4: Verificar schemas

Crear el archivo:

```
sql/check_schemas.sql
```

Contenido:

```
SELECT schema_name  
FROM information_schema.schemata  
WHERE schema_name IN  
(  
    'staging',  
    'core',  
    'mart'  
)  
ORDER BY schema_name;
```

Modificar temporalmente `etl/run_sql_file.py` para ejecutar :

CAPÍTULO 7. HITO 5: CONSTRUCCIÓN DE LA CAPA STAGING EN DUCKDB57

```
run_sql_file(  
  "sql/check_schemas.sql"  
)
```

Resultado esperado:

```
core  
mart  
staging
```

7.7. Paso 5: Crear script de tablas staging

Crear:

```
sql/02_staging_ddl.sql
```

7.8. Paso 6: Crear tabla tipo *cambio_raw*

Agregar:

```
CREATE TABLE IF NOT EXISTS staging.tipo_cambio_raw  
(  
  fecha TEXT,  
  
  ''',  
  moneda TEXT,  
  
  tasa_compra TEXT,  
  
  tasa_venta TEXT,  
  
  archivo_origen TEXT,  
  
  cargado_en TIMESTAMP  
  ''',  
  
);
```

7.9. Paso 7: Crear tabla `balance_bancario_raw`

Agregar:

```
CREATE TABLE IF NOT EXISTS staging.balance_bancario_raw
(
  periodo TEXT,

  '''
  entidad_cod TEXT,

  entidad_nombre TEXT,

  tipo_entidad TEXT,

  activos_totales TEXT,

  cartera_creditos TEXT,

  depositos TEXT,

  patrimonio TEXT,

  resultado_netto TEXT,

  depositos_me TEXT,

  cartera_me TEXT,

  archivo_origen TEXT,

  cargado_en TIMESTAMP
  '''
);
```

7.10. Nota sobre tipos de datos

En staging usaremos columnas tipo TEXT.

La idea es cargar primero los datos crudos y validar después.

Esta decisión permite:

- Evitar fallos tempranos por formatos incorrectos.
- Conservar el dato original.
- Separar carga de transformación.
- Diagnosticar errores con mayor facilidad.

7.11. Paso 8: Ejecutar DDL de staging

Modificar temporalmente:

etl/run_sql_file.py

para ejecutar:

```
run_sql_file(  
    "sql/02_staging_ddl.sql"  
)
```

Luego ejecutar:

```
python etl/run_sql_file.py
```

7.12. Paso 9: Verificar tablas creadas

Crear:

sql/check_staging_tables.sql

Contenido:

```
SELECT table_schema,  
       table_name  
FROM information_schema.tables  
WHERE table_schema = 'staging'  
ORDER BY table_name;
```

Resultado esperado:

```
staging    balance_bancario_raw  
staging    tipo_cambio_raw
```

7.13. Paso 10: Crear script general para inicializar la base

Crear:

etl/init_database.py

Contenido:

```
from db import get_connection

def run_sql(path, conn):

    '''
    with open(path, "r", encoding="utf-8") as file:
        sql = file.read()

    conn.execute(sql)
    '''

def main():

    '''
    conn = get_connection()

    run_sql(
        "sql/01_create_schemas.sql",
        conn
    )

    run_sql(
        "sql/02_staging_ddl.sql",
        conn
    )

    conn.close()

    print("Base DuckDB inicializada correctamente.")
    '''

if __name__ == "__main__":
```

```
main()
```

7.14. Paso 11: Ejecutar inicialización completa

Desde la terminal:

```
python etl/init_database.py
```

Salida esperada:

Base DuckDB inicializada correctamente.

7.15. Paso 12: Crear consulta de monitoreo

Crear:

```
sql/monitor_staging.sql
```

Contenido:

```
SELECT
'tipo_cambio_raw' AS tabla,
COUNT(*) AS total_filas
FROM staging.tipo_cambio_raw
```

```
UNION ALL
```

```
SELECT
'balance_bancario_raw' AS tabla,
COUNT(*) AS total_filas
FROM staging.balance_bancario_raw;
```

Inicialmente ambas tablas deberán tener cero filas.

7.16. Buenas prácticas

Durante este hito seguiremos estas reglas:

1. La base DuckDB vive en data/warehouse.
2. Los schemas separan las capas del Data Warehouse.

3. La capa staging recibe datos crudos.
4. Las transformaciones ocurrirán en core.
5. Las vistas para análisis se construirán en mart.

7.17. Checklist de validación

Antes de continuar verificar:

- ☐ Archivo `macrofinanzas.duckdb` existe.
- ☐ Schema staging creado.
- ☐ Schema core creado.
- ☐ Schema mart creado.
- ☐ Tabla `tipocambiorawcreada`.
- ☐ Tabla `balancebancariorawcreada`.
- ☐ Script `initdatabase.pyejecutado`.
- ☐ Consulta de monitoreo creada.

7.18. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Base DuckDB inicializada.
- Schemas creados.
- Tablas staging creadas.
- Script de inicialización.
- Capa Bronze lista para recibir datos.

7.19. Próximo hito

En el Hito 6 construiremos el proceso ETL de carga.
Aprenderemos a:

- Leer archivos CSV.
- Cargar datos hacia DuckDB.
- Limpiar la capa staging antes de cada carga.
- Automatizar el flujo de ingesta.
- Verificar conteos de registros cargados.

Capítulo 8

Hito 6: Construcción del Primer Proceso ETL

8.1. Objetivo

El objetivo de este hito es construir el primer flujo completo de carga de datos.

Por primera vez conectaremos:

```
CSV
|
V
Python
|
V
PostgreSQL
```

Al finalizar este hito el estudiante será capaz de:

- Leer archivos CSV mediante Python.
- Conectarse a PostgreSQL utilizando SQLAlchemy.
- Cargar información en la capa staging.
- Automatizar el proceso de carga.
- Validar que los datos fueron cargados correctamente.

8.2. Resultado esperado

Al finalizar este hito los datos contenidos en:

`data/raw/tipo_cambio.csv`

`data/raw/balances.csv`

deberán encontrarse cargados en:

`staging.tipo_cambio_raw`

`staging.balance_bancario_raw`

8.3. Arquitectura del proceso

El flujo que construiremos será:

```
CSV
|
V
Pandas
|
V
SQLAlchemy
|
V
PostgreSQL
```

8.4. Paso 1: Verificar archivos fuente

Confirmar la existencia de:

`data/raw/tipo_cambio.csv`

`data/raw/balances.csv`

Verificar que pueden abrirse correctamente.

8.5. Paso 2: Crear módulo de conexión

Verificar el archivo:

etl/db.py

Contenido:

```
from sqlalchemy import create_engine
from dotenv import load_dotenv

import os

load_dotenv()

HOST = os.getenv("DB_HOST")
PORT = os.getenv("DB_PORT")
DB = os.getenv("DB_NAME")
USER = os.getenv("DB_USER")
PASSWORD = os.getenv("DB_PASSWORD")

connection_string = (
    f"postgresql+psycopg2://"
    f"{USER}:{PASSWORD}@{HOST}:{PORT}/{DB}"
)

engine = create_engine(
    connection_string
)
```

8.6. Paso 3: Crear script de carga

Crear:

etl/load_staging.py

8.7. Paso 4: Importar librerías

Agregar:

```
import pandas as pd

from db import engine
```

8.8. Paso 5: Leer tipo de cambio

Agregar:

```
fx = pd.read_csv(
    "data/raw/tipo_cambio.csv"
)

print(
    f"Filas_tipo_cambio: {len(fx)}"
)
```

8.9. Paso 6: Leer balances

Agregar:

```
balances = pd.read_csv(
    "data/raw/balances.csv"
)

print(
    f"Filas_balances: {len(balances)}"
)
```

8.10. Paso 7: Agregar metadatos

Agregar:

```
fx["archivo_origen"] = (
    "tipo_cambio.csv"
)

balances["archivo_origen"] = (
    "balances.csv"
)
```

Estos metadatos permiten rastrear el origen de cada carga.

8.11. Paso 8: Limpiar staging antes de cargar

Crear:

sql/truncate_staging.sql

Contenido:

```
TRUNCATE TABLE
staging.tipo_cambio_raw;

TRUNCATE TABLE
staging.balance_bancario_raw;
```

Esta estrategia garantiza idempotencia.

8.12. Paso 9: Ejecutar truncado desde Python

Agregar:

```
from sqlalchemy import text

with engine.begin() as conn:

    '''
    conn.execute(
        text(
            """
            TRUNCATE TABLE
            staging.tipo_cambio_raw;

            TRUNCATE TABLE
            staging.balance_bancario_raw;
            """
        )
    )
    '''
```

8.13. Paso 10: Cargar tipo de cambio

Agregar:

```
fx.to_sql(  
    "tipo_cambio_raw",  
    con=engine,  
    schema="staging",  
    if_exists="append",  
    index=False  
)
```

8.14. Paso 11: Cargar balances

Agregar:

```
balances.to_sql(  
    "balance_bancario_raw",  
    con=engine,  
    schema="staging",  
    if_exists="append",  
    index=False  
)
```

8.15. Paso 12: Confirmar carga

Agregar:

```
print(  
    "Carga_ completada."  
)
```

8.16. Versión completa del script

El archivo completo debe verse así:

```
import pandas as pd  
  
from sqlalchemy import text  
  
from db import engine  
  
fx = pd.read_csv(  

```

CAPÍTULO 8. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL70

```
"data/raw/tipo_cambio.csv"
)

balances = pd.read_csv(
"data/raw/balances.csv"
)

fx["archivo_origen"] = (
"tipo_cambio.csv"
)

balances["archivo_origen"] = (
"balances.csv"
)

with engine.begin() as conn:

    '''
    conn.execute(
        text(
            """
            TRUNCATE TABLE
            staging.tipo_cambio_raw;

            TRUNCATE TABLE
            staging.balance_bancario_raw;
            """
        )
    )

    '''

fx.to_sql(
"tipo_cambio_raw",
con=engine,
schema="staging",
if_exists="append",
index=False
)

balances.to_sql(
```

CAPÍTULO 8. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL

```
"balance_bancario_raw",
con=engine,
schema="staging",
if_exists="append",
index=False
)

print("Carga completada.")
```

8.17. Paso 13: Ejecutar ETL

Desde terminal:

```
python etl/load_staging.py
```

Salida esperada:

Carga completada.

8.18. Paso 14: Verificar en PostgreSQL

Consultar:

```
SELECT COUNT(*)
FROM staging.tipo_cambio_raw;
```

Consultar:

```
SELECT COUNT(*)
FROM staging.balance_bancario_raw;
```

Los conteos deben coincidir con los CSV.

8.19. Paso 15: Inspeccionar datos

Consultar:

```
SELECT *
FROM staging.tipo_cambio_raw
LIMIT 10;
```

Consultar:

CAPÍTULO 8. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL72

```
SELECT *
FROM staging.balance_bancario_raw
LIMIT 10;
```

8.20. Paso 16: Crear script de validación

Crear:

sql/check_load.sql

Contenido:

```
SELECT
COUNT(*) AS total_fx
FROM staging.tipo_cambio_raw;

SELECT
COUNT(*) AS total_balances
FROM staging.balance_bancario_raw;
```

8.21. Paso 17: Crear orquestador

Crear:

etl/run_pipeline.py

Contenido inicial:

```
import subprocess

subprocess.run(
[
"python",
"etl/load_staging.py"
]
)
```

Este archivo será el punto de entrada de todo el pipeline.

8.22. Buenas prácticas

Durante este proyecto seguiremos las siguientes reglas:

1. El ETL debe ser reproducible.
2. La carga debe ser idempotente.
3. No modificar datos manualmente.
4. Validar cada carga.
5. Mantener trazabilidad mediante metadatos.

8.23. Checklist de validación

Antes de continuar verificar:

- ☐ Python puede leer ambos CSV.
- ☐ Python puede conectarse a PostgreSQL.
- ☐ Staging se limpia correctamente.
- ☐ Los datos son cargados.
- ☐ Los conteos coinciden.
- ☐ El script puede ejecutarse múltiples veces.

8.24. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Script de carga funcional.
- Datos cargados en staging.
- Validaciones básicas.
- Pipeline reproducible.
- Primer flujo ETL completo.

8.25. Próximo hito

En el Hito 7 construiremos la capa Core.
Crearemos:

- Dimensión Tiempo.
- Dimensión Moneda.
- Dimensión Entidad Financiera.
- Claves subrogadas.
- Primera transformación desde staging hacia el modelo dimensional.

Capítulo 9

Hito 6: Construcción del Primer Proceso ETL con DuckDB

9.1. Objetivo

El objetivo de este hito es construir el primer flujo completo de carga de datos utilizando DuckDB.

Por primera vez conectaremos:

```
CSV
|
V
Python
|
V
DuckDB
```

Al finalizar este hito el estudiante será capaz de:

- Leer archivos CSV mediante Pandas.
- Conectarse a DuckDB desde Python.
- Cargar información a la capa staging.
- Automatizar el proceso de carga.
- Validar que los datos fueron cargados correctamente.

9.2. Resultado esperado

Al finalizar este hito los datos contenidos en:

`data/raw/tipo_cambio.csv`

`data/raw/balances.csv`

deberán encontrarse cargados en:

`staging.tipo_cambio_raw`

`staging.balance_bancario_raw`

dentro de DuckDB.

9.3. Arquitectura del proceso

El flujo construido será:

```
CSV
|
V
Pandas
|
V
DuckDB
```

9.4. Paso 1: Verificar archivos fuente

Confirmar la existencia de:

`data/raw/tipo_cambio.csv`

`data/raw/balances.csv`

9.5. Paso 2: Verificar conexión

Verificar:

etl/db.py

Contenido:

```
import duckdb

DB_PATH = (
    "data/warehouse/"
    "macrofinanzas.duckdb"
)

def get_connection():
    """
    return duckdb.connect(
        DB_PATH
    )
    """
```

9.6. Paso 3: Crear script de carga

Crear:

etl/load_staging.py

9.7. Paso 4: Importar librerías

Agregar:

```
import pandas as pd

from db import get_connection
```

9.8. Paso 5: Leer tipo de cambio

Agregar:

```
fx = pd.read_csv(
    "data/raw/tipo_cambio.csv"
)
```

```
print(  
f"Filas_FX: {len(fx)}"  
)
```

9.9. Paso 6: Leer balances

Agregar:

```
balances = pd.read_csv(  
"data/raw/balances.csv"  
)
```

```
print(  
f"Filas_balances: {len(balances)}"  
)
```

9.10. Paso 7: Agregar metadatos

Agregar:

```
fx["archivo_origen"] = (  
"tipo_cambio.csv"  
)
```

```
balances["archivo_origen"] = (  
"balances.csv"  
)
```

Agregar timestamp:

```
from datetime import datetime
```

```
fx["cargado_en"] = (  
datetime.now()  
)
```

```
balances["cargado_en"] = (  
datetime.now()  
)
```

9.11. Paso 8: Conectarse a DuckDB

Agregar:

```
conn = get_connection()
```

9.12. Paso 9: Limpiar staging

Agregar:

```
conn.execute(
    """
    DELETE FROM
    staging.tipo_cambio_raw;
    """
)

conn.execute(
    """
    DELETE FROM
    staging.balance_bancario_raw;
    """
)
```

Esta estrategia garantiza que la carga sea reproducible.

9.13. Paso 10: Registrar DataFrames

DuckDB permite consultar directamente DataFrames.

Agregar:

```
conn.register(
    "fx_df",
    fx
)

conn.register(
    "balances_df",
    balances
)
```


9.14. Paso 11: Cargar tipo de cambio

Agregar:

```
conn.execute(  
    """  
    INSERT INTO  
    staging.tipo_cambio_raw  
  
    SELECT *  
    FROM fx_df;  
    """  
)
```

9.15. Paso 12: Cargar balances

Agregar:

```
conn.execute(  
    """  
    INSERT INTO  
    staging.balance_bancario_raw  
  
    SELECT *  
    FROM balances_df;  
    """  
)
```

9.16. Paso 13: Verificar conteos

Agregar:

```
result_fx = conn.execute(  
    """  
    SELECT COUNT(*)  
    FROM staging.tipo_cambio_raw  
    """  
) .fetchone()[0]  
  
result_balances = conn.execute(  
    """  
    SELECT COUNT(*)  
    FROM staging.balance_bancario_raw  
    """  
) .fetchone()[0]
```

```
"""
SELECT COUNT(*)
FROM staging.balance_bancario_raw
"""
).fetchone()[0]

print(
    f"Filas staging FX: {result_fx}"
)

print(
    f"Filas staging balances: {result_balances}"
)
```

9.17. Paso 14: Cerrar conexión

Agregar:

```
conn.close()
```

9.18. Versión completa del script

```
import pandas as pd

from datetime import datetime

from db import get_connection

fx = pd.read_csv(
    "data/raw/tipo_cambio.csv"
)

balances = pd.read_csv(
    "data/raw/balances.csv"
)

fx["archivo_origen"] = (
    "tipo_cambio.csv"
```

CAPÍTULO 9. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL CON DUCKDB82

```
)

balances["archivo_origen"] = (
    "balances.csv"
)

fx["cargado_en"] = (
    datetime.now()
)

balances["cargado_en"] = (
    datetime.now()
)

conn = get_connection()

conn.execute(
    """
DELETE FROM
staging.tipo_cambio_raw;
    """
)

conn.execute(
    """
DELETE FROM
staging.balance_bancario_raw;
    """
)

conn.register(
    "fx_df",
    fx
)

conn.register(
    "balances_df",
    balances
)
```

CAPÍTULO 9. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL CON DUCKDB83

```
conn.execute(
    """
    INSERT INTO
    staging.tipo_cambio_raw

    SELECT *
    FROM fx_df;
    """
)

conn.execute(
    """
    INSERT INTO
    staging.balance_bancario_raw

    SELECT *
    FROM balances_df;
    """
)

result_fx = conn.execute(
    """
    SELECT COUNT(*)
    FROM staging.tipo_cambio_raw
    """
).fetchone()[0]

result_balances = conn.execute(
    """
    SELECT COUNT(*)
    FROM staging.balance_bancario_raw
    """
).fetchone()[0]

print(
    f"Filas staging_FX: {result_fx}"
)

print(
    f"Filas staging_balances: {result_balances}"
)
```

```
)  
  
conn.close()  
  
print("Carga_ completada.")
```

9.19. Paso 15: Ejecutar ETL

Desde terminal:

```
python etl/load_staging.py
```

Salida esperada:

Filas FX: 100

Filas balances: 50

Filas staging FX: 100

Filas staging balances: 50

Carga completada.

9.20. Paso 16: Crear consulta de monitoreo

Crear:

```
sql/monitor_staging.sql
```

Contenido:

```
SELECT  
'tipo_cambio_raw' AS tabla,  
COUNT(*) AS total_filas  
FROM staging.tipo_cambio_raw  
  
UNION ALL  
  
SELECT  
'balance_bancario_raw' AS tabla,  
COUNT(*) AS total_filas  
FROM staging.balance_bancario_raw;
```

9.21. Paso 17: Ejecutar consulta

Desde Python:

```
result = conn.execute(  
    open(  
        "sql/monitor_staging.sql"  
    ).read()  
).fetchdf()  
  
print(result)
```

Resultado esperado:

tabla	total_filas
0 tipo_cambio_raw	100
1 balance_bancario_raw	50

9.22. Paso 18: Crear orquestador

Crear:

etl/run_pipeline.py

Contenido inicial:

```
import subprocess  
  
subprocess.run(  
    [  
        "python",  
        "etl/load_staging.py"  
    ],  
    check=True  
)  
  
print(  
    "Pipeline ejecutado correctamente."  
)
```

9.23. Buenas prácticas

Durante este hito seguiremos las siguientes reglas:

1. Todas las cargas deben ser reproducibles.
2. No modificar tablas staging manualmente.
3. Mantener metadatos de carga.
4. Validar conteos después de cada carga.
5. Cerrar conexiones al finalizar.

9.24. Checklist de validación

- ☐ CSV leídos correctamente.
- ☐ Conexión DuckDB funcional.
- ☐ Tablas staging limpiadas.
- ☐ Datos cargados.
- ☐ Conteos validados.
- ☐ Script `monitor_staging.sql` ejecutado.
- ☐ Pipeline reproducible.

9.25. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Primer proceso ETL funcional.
- Datos cargados en staging.
- Consulta de monitoreo.
- Orquestador inicial.
- Flujo completo CSV → Python → DuckDB.

9.26. Próximo hito

En el Hito 7 construiremos la capa Core.
Crearemos:

- Dimensión Tiempo.
- Dimensión Moneda.
- Dimensión Entidad Financiera.
- Claves subrogadas.
- Primera transformación desde staging hacia el modelo dimensional.

Capítulo 10

Hito 7: Construcción de la Capa Core y Dimensiones en DuckDB

10.1. Objetivo

El objetivo de este hito es construir la capa core del Data Warehouse.

En esta capa comenzaremos a transformar los datos crudos almacenados en staging en estructuras analíticas organizadas siguiendo principios de modelado dimensional.

Al finalizar este hito el estudiante será capaz de:

- Crear dimensiones en DuckDB.
- Comprender el uso de claves subrogadas.
- Construir una dimensión tiempo.
- Construir una dimensión moneda.
- Construir una dimensión entidad financiera.
- Poblar dimensiones a partir de los datos de staging.
- Preparar la base para construir las tablas de hechos.

10.2. Resultado esperado

Al finalizar este hito deberán existir:

`core.dim_tiempo`

`core.dim_moneda`

`core.dim_entidad_financiera`

Estas dimensiones serán utilizadas posteriormente por las tablas de hechos.

10.3. Arquitectura del hito

El flujo de transformación será:

staging

|

V

`core.dim_tiempo`

`core.dim_moneda`

`core.dim_entidad_financiera`

10.4. Paso 1: Crear script de dimensiones

Crear:

`sql/03_dimensions_ddl.sql`

10.5. Paso 2: Crear dimensión tiempo

Agregar:

```
CREATE TABLE IF NOT EXISTS core.dim_tiempo
(  
  sk_tiempo INTEGER PRIMARY KEY,
```

CAPÍTULO 10. HITO 7: CONSTRUCCIÓN DE LA CAPA CORE Y DIMENSIONES EN DUCKDB90

```
'''
fecha DATE NOT NULL,

anio INTEGER,

mes INTEGER,

trimestre INTEGER,

anio_mes INTEGER,

nombre_mes VARCHAR,

es_fin_de_mes BOOLEAN
'''

);
```

10.6. Paso 3: Crear dimensión moneda

Agregar:

```
CREATE TABLE IF NOT EXISTS core.dim_moneda
(
sk_moneda INTEGER PRIMARY KEY,

'''
moneda_cod VARCHAR,

moneda_nombre VARCHAR
'''

);
```

10.7. Paso 4: Crear dimensión entidad financiera

Agregar:

```
CREATE TABLE IF NOT EXISTS
```

```
core.dim_entidad_financiera
(
sk_entidad INTEGER PRIMARY KEY,

'''
entidad_cod VARCHAR,

entidad_nombre VARCHAR,

tipo_entidad VARCHAR,

vigente_desde DATE,

vigente_hasta DATE,

es_actual BOOLEAN
'''

);
```

10.8. Paso 5: Ejecutar DDL

Desde Python:

```
from db import get_connection

conn = get_connection()

with open(
    "sql/03_dimensions_ddl.sql",
    "r",
    encoding="utf-8"
) as file:

    '''
    conn.execute(
        file.read()
    )
    '''
```

```
conn.close()
```

10.9. Paso 6: Crear script de carga

Crear:

```
sql/04_load_dimensions.sql
```

10.10. Paso 7: Limpiar dimensiones

Agregar:

```
DELETE FROM  
core.dim_entidad_financiera;
```

```
DELETE FROM  
core.dim_moneda;
```

```
DELETE FROM  
core.dim_tiempo;
```

10.11. Paso 8: Poblar dimensión tiempo

Agregar:

```
INSERT INTO core.dim_tiempo
```

```
SELECT
```

```
‘‘‘
```

```
CAST(  
    strftime(  
        fecha,  
        '%Y%m%d'  
    ) AS INTEGER  
) AS sk_tiempo,
```

```
fecha,
```

```
year(fecha) AS anio,

month(fecha) AS mes,

quarter(fecha) AS trimestre,

CAST(
    strftime(
        fecha,
        '%Y%m'
    ) AS INTEGER
) AS anio_mes,

monthname(fecha)
    AS nombre_mes,

fecha =
last_day(fecha)
    AS es_fin_de_mes
'',

FROM generate_series(
DATE '2022-01-01',
DATE '2024-12-31',
INTERVAL 1 DAY
) t(fecha);
```

10.12. Explicación

DuckDB posee una función muy útil:

```
generate_series()
```

que permite generar calendarios completos sin necesidad de archivos externos.

10.13. Paso 9: Poblar dimensión moneda

Agregar:

CAPÍTULO 10. HITO 7: CONSTRUCCIÓN DE LA CAPA CORE Y DIMENSIONES EN DUCKDB94

```
INSERT INTO core.dim_moneda

SELECT

'''
ROW_NUMBER()
OVER(
    ORDER BY moneda
) AS sk_moneda,

moneda AS moneda_cod,

CASE

    WHEN moneda='USD'
    THEN 'Dolar_estadounidense'

    WHEN moneda='EUR'
    THEN 'Euro'

    ELSE moneda

END AS moneda_nombre
'''

FROM
(
SELECT DISTINCT
moneda
FROM staging.tipo_cambio_raw
) x;
```

10.14. Paso 10: Poblar dimensión entidad financiera

Agregar:

```
INSERT INTO
core.dim_entidad_financiera
```

CAPÍTULO 10. HITO 7: CONSTRUCCIÓN DE LA CAPA CORE Y DIMENSIONES EN DUCKDB95

```
SELECT

    '','','
ROW_NUMBER()
OVER(
    ORDER BY entidad_cod
) AS sk_entidad,

entidad_cod,

entidad_nombre,

tipo_entidad,

DATE '2022-01-01'
    AS vigente_desde,

DATE '9999-12-31'
    AS vigente_hasta,

TRUE
    AS es_actual
    '','','

FROM
(
SELECT DISTINCT

    '','','
    entidad_cod,

    entidad_nombre,

    tipo_entidad

FROM
    staging.balance_bancario_raw
    '','','
```



```
) x;
```

10.15. Nota sobre SCD Tipo 2

En esta primera versión no implementaremos todavía una dimensión histórica completa.

La lógica será:

- Una fila por entidad.
- Una sola versión vigente.
- Sin cambios históricos.

En una versión posterior agregaremos:

- Versionamiento.
- Fechas de vigencia.
- SCD Tipo 2 completo.

10.16. Paso 11: Ejecutar carga

Crear:

```
etl/load_dimensions.py
```

Contenido:

```
from db import get_connection

conn = get_connection()

with open(
    "sql/04_load_dimensions.sql",
    "r",
    encoding="utf-8"
) as file:

    '''
```

```
sql = file.read()
'''

conn.execute(sql)

conn.close()

print(
    "Dimensiones cargadas."
)
```

10.17. Paso 12: Verificar dimensión tiempo

Crear:

sql/check_dim_tiempo.sql

```
SELECT *
FROM core.dim_tiempo
LIMIT 10;
```

10.18. Paso 13: Verificar dimensión moneda

```
SELECT *
FROM core.dim_moneda;
```

Resultado esperado:

```
USD
EUR
```

10.19. Paso 14: Verificar dimensión entidad financiera

```
SELECT *
FROM core.dim_entidad_financiera;
```

10.20. Paso 15: Crear consulta de monitoreo

Crear:

sql/check_dimensions.sql

Contenido:

```
SELECT

'''
'dim_tiempo' AS tabla,

COUNT(*) AS total_filas
'''

FROM core.dim_tiempo

UNION ALL

SELECT

'''
'dim_moneda',

COUNT(*)
'''

FROM core.dim_moneda

UNION ALL

SELECT

'''
'dim_entidad_financiera',

COUNT(*)
'''

FROM core.dim_entidad_financiera;
```

10.21. Paso 16: Ejecutar monitoreo

Resultado esperado:

```
dim_tiempo  
  
dim_moneda  
  
dim_entidad_financiera  
  
con conteos positivos.
```

10.22. Paso 17: Integrar al pipeline

Actualizar:

```
etl/run_pipeline.py  
  
import subprocess  
  
subprocess.run(  
    [  
        "python",  
        "etl/load_staging.py"  
    ],  
    check=True  
)  
  
subprocess.run(  
    [  
        "python",  
        "etl/load_dimensions.py"  
    ],  
    check=True  
)  
  
print(  
    "Pipeline_ejecutado_correctamente."  
)
```

10.23. Buenas prácticas

Durante este hito seguiremos las siguientes reglas:

1. Las dimensiones siempre se construyen desde staging.
2. Las claves subrogadas pertenecen al Data Warehouse.
3. La dimensión tiempo debe ser reutilizable.
4. La dimensión entidad debe ser preparada para soportar SCD.
5. Las cargas deben ser reproducibles.

10.24. Checklist de validación

- ☐ Dimensión tiempo creada.
- ☐ Dimensión moneda creada.
- ☐ Dimensión entidad creada.
- ☐ Dimensión tiempo poblada.
- ☐ Dimensión moneda poblada.
- ☐ Dimensión entidad poblada.
- ☐ Script de monitoreo creado.
- ☐ Pipeline actualizado.

10.25. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Tres dimensiones pobladas.
- Scripts SQL documentados.
- Script de carga automatizado.
- Pipeline funcional.
- Capa Core parcialmente construida.

10.26. Próximo hito

En el Hito 8 construiremos las tablas de hechos.
Aprenderemos a:

- Resolver claves subrogadas.
- Crear $\text{fact}_{\text{tipo}_{\text{ambio}}}$.
- Crear $\text{fact}_{\text{balance}_{\text{mensual}}}$.
- Calcular medidas derivadas.
- Validar el grano de cada hecho.

Capítulo 11

Hito 8: Construcción de las Tablas de Hechos en DuckDB

11.1. Objetivo

El objetivo de este hito es construir las tablas de hechos del Data Warehouse macrofinanciero utilizando DuckDB.

Las tablas de hechos almacenan las medidas numéricas del modelo dimensional. En este proyecto construiremos dos hechos principales:

- $\text{fact}_{t\text{ipo}_c\text{ambio}}$
- $\text{fact}_{b\text{alance}_m\text{ensual}}$

Al finalizar este hito el estudiante será capaz de:

- Crear tablas de hechos en DuckDB.
- Cargar hechos desde la capa staging.
- Resolver claves subrogadas utilizando las dimensiones.
- Calcular medidas derivadas.
- Validar el grano de cada tabla de hechos.
- Preparar la información para construir data marts.

11.2. Resultado esperado

Al finalizar este hito deberán existir:

`core.fact_tipo_cambio`

`core.fact_balance_mensual`

Estas tablas deberán estar conectadas lógicamente con:

`core.dim_tiempo`

`core.dim_moneda`

`core.dim_entidad_financiera`

11.3. Arquitectura del hito

El flujo será:

`staging.tipo_cambio_raw`

|

V

`core.fact_tipo_cambio`

`staging.balance_bancario_raw`

|

V

`core.fact_balance_mensual`

11.4. Paso 1: Crear script DDL de hechos

Crear:

`sql/05_facts_ddl.sql`

11.5. Paso 2: Crear tabla `facttipocambio`

Agregar:


```
CREATE TABLE IF NOT EXISTS core.fact_tipo_cambio
(
  sk_tiempo INTEGER,

  '''
  sk_moneda INTEGER,

  tasa_compra DOUBLE,

  tasa_venta DOUBLE,

  tasa_promedio DOUBLE,

  spread DOUBLE
  '''
);
```

11.6. Paso 3: Crear tabla *fact_{balance} mensual*

Agregar:

```
CREATE TABLE IF NOT EXISTS core.fact_balance_mensual
(
  sk_tiempo INTEGER,

  '''
  sk_entidad INTEGER,

  activos_totales DOUBLE,

  cartera_creditos DOUBLE,

  depositos DOUBLE,

  patrimonio DOUBLE,

  resultado_netto DOUBLE,

  depositos_me DOUBLE,
```

```
cartera_me DOUBLE
'''

);
```

11.7. Nota sobre restricciones

DuckDB permite definir llaves primarias y restricciones, pero para este primer proyecto mantendremos el diseño simple. La validación del grano se realizará mediante consultas de control.

11.8. Paso 4: Crear script de carga de hechos

Crear:

```
sql/06_load_facts.sql
```

11.9. Paso 5: Limpiar hechos

Agregar:

```
DELETE FROM core.fact_tipo_cambio;

DELETE FROM core.fact_balance_mensual;
```

11.10. Paso 6: Cargar $fact_{tipo_cambio}$

Agregar:

```
INSERT INTO core.fact_tipo_cambio
(
  sk_tiempo,
  sk_moneda,
  tasa_compra,
  tasa_venta,
  tasa_promedio,
  spread
)
```

```
SELECT

'''
t.sk_tiempo,

m.sk_moneda,

CAST(r.tasa_compra AS DOUBLE)
    AS tasa_compra,

CAST(r.tasa_venta AS DOUBLE)
    AS tasa_venta,

(
    CAST(r.tasa_compra AS DOUBLE)
    +
    CAST(r.tasa_venta AS DOUBLE)
) / 2
    AS tasa_promedio,

(
    CAST(r.tasa_venta AS DOUBLE)
    -
    CAST(r.tasa_compra AS DOUBLE)
)
    AS spread
'''

FROM staging.tipo_cambio_raw r

JOIN core.dim_tiempo t
ON t.fecha = CAST(r.fecha AS DATE)

JOIN core.dim_moneda m
ON m.moneda_cod = r.moneda

WHERE r.fecha IS NOT NULL
AND r.moneda IS NOT NULL
AND CAST(r.tasa_compra AS DOUBLE) > 0
```

```
AND CAST(r.tasa_venta AS DOUBLE) > 0;
```

11.11. Explicación

En esta transformación:

- Se convierte la fecha desde texto hacia DATE.
- Se convierten las tasas desde texto hacia DOUBLE.
- Se resuelve la clave sk_{tiempo} .
- Se resuelve la clave sk_{moneda} .
- Se calcula $tasa_{promedio}$.
- Se calcula spread.

11.12. Paso 7: Cargar $fact_{balance_{mensual}}$

Agregar:

```
INSERT INTO core.fact_balance_mensual
(
  sk_tiempo,
  sk_entidad,
  activos_totales,
  cartera_creditos,
  depositos,
  patrimonio,
  resultado_netto,
  depositos_me,
  cartera_me
)
```

```
SELECT
```

```
'''
```

```
t.sk_tiempo,
```

```
e.sk_entidad,
```

CAPÍTULO 11. HITO 8: CONSTRUCCIÓN DE LAS TABLAS DE HECHOS EN DUCKDB108

```
CAST(r.activos_totales AS DOUBLE)
    AS activos_totales,

CAST(r.cartera_creditos AS DOUBLE)
    AS cartera_creditos,

CAST(r.depositos AS DOUBLE)
    AS depositos,

CAST(r.patrimonio AS DOUBLE)
    AS patrimonio,

CAST(r.resultado_neto AS DOUBLE)
    AS resultado_neto,

CAST(r.depositos_me AS DOUBLE)
    AS depositos_me,

CAST(r.cartera_me AS DOUBLE)
    AS cartera_me
,,,

FROM staging.balance_bancario_raw r

JOIN core.dim_tiempo t
ON t.fecha =
last_day(
CAST(
r.periodo || '01'
AS DATE
)
)

JOIN core.dim_entidad_financiera e
ON e.entidad_cod = r.entidad_cod

WHERE r.periodo IS NOT NULL
AND r.entidad_cod IS NOT NULL;
```

11.13. Advertencia sobre fechas mensuales

Si el campo periodo viene con formato YYYYMM, DuckDB puede requerir una conversión explícita.

Si el bloque anterior falla, utilizar esta versión alternativa:

```
STRPTIME(
r.periodo || '01',
'%Y%m%d'
)::DATE
```

La unión con *dim_tiempo* quedara:

```
JOIN core.dim_tiempo t
ON t.fecha =
last_day(
STRPTIME(
r.periodo || '01',
'%Y%m%d'
)::DATE
)
```

11.14. Paso 8: Crear script Python para cargar hechos

Crear:

etl/load_facts.py

Contenido:

```
from db import get_connection

def run_sql_file(path, conn):
    '''
    with open(
        path,
        "r",
        encoding="utf-8"
    ) as file:
```

```
        sql = file.read()

conn.execute(sql)
'''

def main():

    '''
    conn = get_connection()

    run_sql_file(
        "sql/05_facts_ddl.sql",
        conn
    )

    run_sql_file(
        "sql/06_load_facts.sql",
        conn
    )

    conn.close()

    print(
        "Hechos cargados correctamente."
    )
    '''

if __name__ == "__main__":
    main()
```

11.15. Paso 9: Ejecutar carga de hechos

Desde la terminal:

```
python etl/load_facts.py
```

Salida esperada:

Hechos cargados correctamente.

11.16. Paso 10: Crear consulta de monitoreo

Crear:

sql/check_facts.sql

Contenido:

```
SELECT
'fact_tipo_cambio' AS tabla,
COUNT(*) AS total_filas
FROM core.fact_tipo_cambio

UNION ALL

SELECT
'fact_balance_mensual' AS tabla,
COUNT(*) AS total_filas
FROM core.fact_balance_mensual;
```

11.17. Paso 11: Crear script para revisar hechos

Crear:

etl/check_facts.py

Contenido:

```
from db import get_connection

conn = get_connection()

with open(
    "sql/check_facts.sql",
    "r",
    encoding="utf-8"
) as file:

    '''
    sql = file.read()
    '''
```



```
result = conn.execute(sql).fetchdf()

print(result)

conn.close()

Ejecutar:
python etl/check_facts.py
```

11.18. Paso 12: Inspeccionar $fact_{tipo_cambio}$

```
Crear:

sql/inspect_fact_tipo_cambio.sql

Contenido:

SELECT

'''
t.fecha,

m.moneda_cod,

f.tasa_compra,

f.tasa_venta,

f.tasa_promedio,

f.spread
'''

FROM core.fact_tipo_cambio f

JOIN core.dim_tiempo t
ON t.sk_tiempo = f.sk_tiempo

JOIN core.dim_moneda m
```

```
ON m.sk_moneda = f.sk_moneda
```

```
ORDER BY  
t.fecha,  
m.moneda_cod
```

```
LIMIT 10;
```

11.19. Paso 13: Inspeccionar $\text{fact}_{balance_mensual}$

Crear:

```
sql/inspect_fact_balance_mensual.sql
```

Contenido:

```
SELECT  
  
'''  
t.fecha,  
  
e.entidad_cod,  
  
e.entidad_nombre,  
  
f.activos_totales,  
  
f.depositos,  
  
f.patrimonio,  
  
f.resultado_netos  
'''  
  
FROM core.fact_balance_mensual f  
  
JOIN core.dim_tiempo t  
ON t.sk_tiempo = f.sk_tiempo  
  
JOIN core.dim_entidad_financiera e
```

```
ON e.sk_entidad = f.sk_entidad

ORDER BY
t.fecha,
e.entidad_cod

LIMIT 10;
```

11.20. Paso 14: Validar el grano de tipo de cambio

Crear:

sql/check_grain_fx.sql

Contenido:

```
SELECT

'''
sk_tiempo,

sk_moneda,

COUNT(*) AS total
'''

FROM core.fact_tipo_cambio

GROUP BY
sk_tiempo,
sk_moneda

HAVING COUNT(*) > 1;
```

Resultado esperado:

0 filas

11.21. Paso 15: Validar el grano de balance mensual

Crear:

sql/check_grain_balance.sql

Contenido:

```
SELECT

'''
sk_tiempo ,

sk_entidad ,

COUNT(*) AS total
'''

FROM core.fact_balance_mensual

GROUP BY
sk_tiempo ,
sk_entidad

HAVING COUNT(*) > 1;
```

Resultado esperado:

0 filas

11.22. Paso 16: Validar medidas derivadas

Crear:

sql/check_fx_measures.sql

Contenido:

```
SELECT

'''
tasa_compra ,

tasa_venta ,

tasa_promedio ,
```

```
spread,

(tasa_compra + tasa_venta) / 2
    AS tasa_promedio_recalculada,

tasa_venta - tasa_compra
    AS spread_recalculado
'''

FROM core.fact_tipo_cambio

LIMIT 10;
```

11.23. Paso 17: Validar reglas de negocio

Crear:

sql/check_balance_business_rules.sql

Contenido:

```
SELECT *
FROM core.fact_balance_mensual
WHERE depositos_me > depositos;

SELECT *
FROM core.fact_balance_mensual
WHERE cartera_me > cartera_creditos;
```

Resultado esperado:

0 filas

11.24. Paso 18: Integrar al pipeline

Actualizar:

etl/run_pipeline.py

Contenido recomendado:

```
import sys

import subprocess

steps = [
    "etl/load_staging.py",
    "etl/load_dimensions.py",
    "etl/load_facts.py"
]

for step in steps:

    '''
    print(
        f"Ejecutando_{step}..."
    )

    subprocess.run(
        [
            sys.executable,
            step
        ],
        check=True
    )
    '''

    print(
        "Pipeline_ejecutado_correctamente."
    )
```

11.25. Buenas prácticas

Durante este hito seguiremos estas reglas:

1. Los hechos deben respetar el grano definido.
2. Los hechos deben usar claves subrogadas.
3. Las medidas derivadas deben calcularse de forma reproducible.
4. Las validaciones deben ejecutarse después de cada carga.

5. El pipeline debe poder ejecutarse de inicio a fin.

11.26. Checklist de validación

- ☐ Tabla $\text{fact}_{\text{tipo}_c\text{ambiocreceda}}$.
- ☐ Tabla $\text{fact}_{\text{balance}_m\text{mensualcreada}}$.
- ☐ Hechos cargados.
- ☐ Conteos revisados.
- ☐ Grano de tipo de cambio validado.
- ☐ Grano de balance mensual validado.
- ☐ Medidas derivadas verificadas.
- ☐ Pipeline actualizado.

11.27. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Tablas de hechos en core.
- Scripts SQL de creación y carga.
- Consultas de inspección.
- Validaciones de grano.
- Modelo dimensional funcional.

11.28. Próximo hito

En el Hito 9 construiremos la capa mart.
Aprenderemos a:

- Crear vistas analíticas.
- Calcular indicadores macrofinancieros.

*CAPÍTULO 11. HITO 8: CONSTRUCCIÓN DE LAS TABLAS DE HECHOS EN DUCKDB*¹¹⁹

- Construir ROA, ROE, dolarización y HHI.
- Preparar datos listos para Streamlit.

Capítulo 12

Hito 9: Construcción de la Capa Mart e Indicadores Analíticos en DuckDB

12.1. Objetivo

El objetivo de este hito es construir la capa mart del Data Warehouse macrofinanciero.

La capa mart contiene vistas analíticas listas para ser consumidas por usuarios finales, reportes y dashboards. A diferencia de staging, que almacena datos crudos, y core, que contiene el modelo dimensional, la capa mart entrega indicadores directamente interpretables.

Al finalizar este hito el estudiante será capaz de:

- Crear vistas analíticas en DuckDB.
- Calcular indicadores macrofinancieros.
- Construir métricas como ROA, ROE, dolarización y spread cambiario.
- Preparar tablas listas para Streamlit.
- Integrar la construcción de marts al pipeline.

12.2. Resultado esperado

Al finalizar este hito deberán existir las siguientes vistas:

CAPÍTULO 12. HITO 9: CONSTRUCCIÓN DE LA CAPA MART E INDICADORES ANALÍTICOS E

`mart.vw_tipo_cambio_diario`

`mart.vw_indicadores_banca`

`mart.vw_hhi_bancario`

`mart.vw_resumen_macrofinanciero`

Estas vistas serán la base del dashboard.

12.3. Arquitectura del hito

El flujo será:

```
core.fact_tipo_cambio
core.fact_balance_mensual
core.dim_tiempo
core.dim_moneda
core.dim_entidad_financiera
|
V
mart
|
V
Streamlit
```

12.4. Paso 1: Crear script de marts

Crear:

`sql/07_marts.sql`

Este archivo contendrá todas las vistas analíticas.

12.5. Paso 2: Crear vista de tipo de cambio diario

Agregar:

CAPÍTULO 12. HITO 9: CONSTRUCCIÓN DE LA CAPA MART E INDICADORES ANALÍTICOS E

```
CREATE OR REPLACE VIEW mart.vw_tipo_cambio_diario AS

SELECT
t.fecha,

'''
t.anio,

t.mes,

t.anio_mes,

m.moneda_cod,

m.moneda_nombre,

f.tasa_compra,

f.tasa_venta,

f.tasa_promedio,

f.spread
'''

FROM core.fact_tipo_cambio f

JOIN core.dim_tiempo t
ON t.sk_tiempo = f.sk_tiempo

JOIN core.dim_moneda m
ON m.sk_moneda = f.sk_moneda;
```

12.6. Explicación

Esta vista transforma la tabla de hechos cambiaria en una tabla amigable para análisis.

En lugar de mostrar claves como:

sk_tiempo

CAPÍTULO 12. HITO 9: CONSTRUCCIÓN DE LA CAPA MART E INDICADORES ANALÍTICOS E

sk_moneda

muestra variables interpretables:

fecha

anio_mes

moneda_cod

tasa_promedio

spread

12.7. Paso 3: Crear vista de indicadores bancarios

Agregar:

```
CREATE OR REPLACE VIEW mart.vw_indicadores_banca AS
```

```
SELECT
```

```
t.fecha,
```

```
'''
```

```
t.anio,
```

```
t.mes,
```

```
t.anio_mes,
```

```
e.entidad_cod,
```

```
e.entidad_nombre,
```

```
e.tipo_entidad,
```

```
f.activos_totales,
```

```
f.cartera_creditos,
```

```
f.depositos,
```

```
f.patrimonio,
```

```

f.resultado_netto ,

f.depositos_me ,

f.cartera_me ,

f.resultado_netto
  / NULLIF(f.activos_totales , 0)
  AS roa ,

f.resultado_netto
  / NULLIF(f.patrimonio , 0)
  AS roe ,

f.depositos_me
  / NULLIF(f.depositos , 0)
  AS dolarizacion_depositos ,

f.cartera_me
  / NULLIF(f.cartera_creditos , 0)
  AS dolarizacion_cartera
'''

FROM core.fact_balance_mensual f

JOIN core.dim_tiempo t
ON t.sk_tiempo = f.sk_tiempo

JOIN core.dim_entidad_financiera e
ON e.sk_entidad = f.sk_entidad;

```

12.8. Indicadores calculados

La vista anterior calcula:

- roa: retorno sobre activos.
- roe: retorno sobre patrimonio.
- dolarizacion_{depositos} : *proporción de depósitos en moneda extranjera.*

- $dolarizacion_{cartera}$: *proporción de cartera en moneda extranjera.*

12.9. Paso 4: Crear vista de concentración bancaria

Agregar:

```
CREATE OR REPLACE VIEW mart.vw_hhi_bancario AS

WITH cuotas AS
(
SELECT
t.fecha,

'''
    t.anio_mes,

    e.entidad_cod,

    e.entidad_nombre,

    f.activos_totales,

    f.activos_totales
      / NULLIF(
        SUM(f.activos_totales)
        OVER (
          PARTITION BY t.anio_mes
        ),
        0
      )
    AS cuota_activos

FROM core.fact_balance_mensual f

JOIN core.dim_tiempo t
    ON t.sk_tiempo = f.sk_tiempo

JOIN core.dim_entidad_financiera e
    ON e.sk_entidad = f.sk_entidad
'''
```

```
)

SELECT
fecha,

'''
anio_mes,

SUM(
    POWER(cuota_activos, 2)
) AS hhi_activos
'''

FROM cuotas

GROUP BY
fecha,
anio_mes

ORDER BY
anio_mes;
```

12.10. Explicación

El índice HHI mide concentración.

Si una sola entidad domina el mercado, el HHI se acerca a 1.

Si el mercado está muy diversificado, el HHI se acerca a 0.

En este proyecto lo calculamos usando participación en activos.

12.11. Paso 5: Crear vista resumen macrofinanciero

Agregar:

```
CREATE OR REPLACE VIEW mart.vw_resumen_macrofinanciero AS

WITH fx AS
(
SELECT
```

CAPÍTULO 12. HITO 9: CONSTRUCCIÓN DE LA CAPA MART E INDICADORES ANALÍTICOS E

```
anio_mes ,

'''
    AVG(tasa_promedio)
        AS tasa_promedio_mes ,

    AVG(spread)
        AS spread_promedio_mes

FROM mart.vw_tipo_cambio_diario

WHERE moneda_cod = 'USD'

GROUP BY
    anio_mes
'''

),

banca AS
(
SELECT
    anio_mes ,

    '''
        SUM(activos_totales)
            AS activos_totales_sistema ,

        SUM(depositos)
            AS depositos_sistema ,

        SUM(resultado_neto)
            AS resultado_neto_sistema ,

        SUM(depositos_me)
            / NULLIF(SUM(depositos), 0)
            AS dolarizacion_depositos_sistema

FROM mart.vw_indicadores_banca
```


CAPÍTULO 12. HITO 9: CONSTRUCCIÓN DE LA CAPA MART E INDICADORES ANALÍTICOS E

```
GROUP BY
    anio_mes
'''

)

SELECT
b.anio_mes ,

'''
fx.tasa_promedio_mes ,

fx.spread_promedio_mes ,

b.activos_totales_sistema ,

b.depositos_sistema ,

b.resultado_neto_sistema ,

b.dolarizacion_depositos_sistema
'''

FROM banca b

LEFT JOIN fx
ON fx.anio_mes = b.anio_mes

ORDER BY
b.anio_mes;
```

12.12. Explicación

Esta vista integra información cambiaria y bancaria en un solo objeto analítico.

Será útil para construir la página principal del dashboard.

12.13. Paso 6: Crear script Python para construir marts

Crear:

etl/load_marts.py

Contenido:

```
from db import get_connection

def run_sql_file(path, conn):

    '''
    with open(
        path,
        "r",
        encoding="utf-8"
    ) as file:

        sql = file.read()

    conn.execute(sql)
    '''

def main():

    '''
    conn = get_connection()

    run_sql_file(
        "sql/07_marts.sql",
        conn
    )

    conn.close()

    print(
        "Marts creados correctamente."
    )
    '''
```

```
if **name** == "**main**":  
    main()
```

12.14. Paso 7: Ejecutar construcción de marts

Desde la terminal:

```
python etl/load_marts.py
```

Salida esperada:

Marts creados correctamente.

12.15. Paso 8: Crear consulta de monitoreo de marts

Crear:

```
sql/check_marts.sql
```

Contenido:

```
SELECT  
table_schema ,  
  
'''  
table_name  
'''  
  
FROM information_schema.tables  
  
WHERE table_schema = 'mart'  
  
ORDER BY  
table_name;
```

12.16. Paso 9: Crear script para revisar marts

Crear:

```
etl/check_marts.py
```

CAPÍTULO 12. HITO 9: CONSTRUCCIÓN DE LA CAPA MART E INDICADORES ANALÍTICOS E

Contenido:

```
from db import get_connection

conn = get_connection()

with open(
    "sql/check_marts.sql",
    "r",
    encoding="utf-8"
) as file:

    '''
    sql = file.read()
    '''

result = conn.execute(sql).fetchdf()

print(result)

conn.close()

Ejecutar:
python etl/check_marts.py
```

12.17. Paso 10: Inspeccionar vista de tipo de cambio

Crear:

```
sql/inspect_vw_tipo_cambio_diario.sql
```

Contenido:

```
SELECT *
FROM mart.vw_tipo_cambio_diario
ORDER BY
fecha,
moneda_cod
LIMIT 10;
```

12.18. Paso 11: Inspeccionar vista bancaria

Crear:

sql/inspect_vw_indicadores_banca.sql

Contenido:

```
SELECT *
FROM mart.vw_indicadores_banca
ORDER BY
fecha,
entidad_cod
LIMIT 10;
```

12.19. Paso 12: Inspeccionar resumen macrofinanciero

Crear:

sql/inspect_vw_resumen_macrofinanciero.sql

Contenido:

```
SELECT *
FROM mart.vw_resumen_macrofinanciero
ORDER BY
anio_mes;
```

12.20. Paso 13: Integrar al pipeline

Actualizar:

etl/run_pipeline.py

Contenido recomendado:

```
import sys

import subprocess
```

```
steps = [  
    "etl/load_staging.py",  
    "etl/load_dimensions.py",  
    "etl/load_facts.py",  
    "etl/load_marts.py"  
]  
  
for step in steps:  
  
    '''  
    print(  
        f"Ejecutando_{step}..."  
    )  
  
    subprocess.run(  
        [  
            sys.executable,  
            step  
        ],  
        check=True  
    )  
    '''  
  
    print(  
        "Pipeline_ejecutado_correctamente."  
    )
```

12.21. Paso 14: Ejecutar pipeline completo

Desde la terminal:

```
python etl/run_pipeline.py
```

Salida esperada:

```
Ejecutando etl/load_staging.py...  
Ejecutando etl/load_dimensions.py...  
Ejecutando etl/load_facts.py...  
Ejecutando etl/load_marts.py...
```

```
Pipeline ejecutado correctamente.
```

12.22. Buenas prácticas

Durante este hito seguiremos estas reglas:

1. El dashboard deberá conectarse a la capa mart, no directamente a core.
2. Los indicadores deben calcularse de forma reproducible.
3. Las vistas deben tener nombres claros.
4. Cada vista debe responder a una pregunta de negocio.
5. Las vistas deben ser fáciles de consumir desde Streamlit.

12.23. Checklist de validación

- ☐ Vista $vw_{tipo_cambio_diari}$ creada.
- ☐ Vista $vw_{indicadores_banc}$ creada.
- ☐ Vista $vw_{hi_bancari}$ creada.
- ☐ Vista $vw_{resumen_macrofinanciero}$ creada.
- ☐ Indicadores bancarios calculados.
- ☐ Resumen macrofinanciero generado.
- ☐ Pipeline actualizado.

12.24. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Capa mart construida.
- Vistas analíticas para Streamlit.
- Indicadores macrofinancieros.
- Pipeline completo hasta data marts.
- Base lista para visualización.

12.25. Próximo hito

En el Hito 10 construiremos el dashboard en Streamlit.
Aprenderemos a:

- Conectar Streamlit con DuckDB.
- Leer vistas de la capa mart.
- Construir KPIs.
- Crear gráficos interactivos.
- Diseñar una primera aplicación macrofinanciera.

Capítulo 13

Hito 10: Construcción del Dashboard en Streamlit

13.1. Objetivo

El objetivo de este hito es construir la primera versión del dashboard macrofinanciero utilizando Streamlit.

Hasta este momento hemos construido:

```
CSV
|
V
Python
|
V
DuckDB
|
V
staging
|
V
core
|
V
mart
```

En este hito conectaremos la capa mart con una aplicación web interactiva.

Al finalizar este hito el estudiante será capaz de:

- Instalar Streamlit.
- Conectar Streamlit con DuckDB.
- Consultar vistas de la capa mart.
- Crear indicadores KPI.
- Crear gráficos básicos.
- Construir una primera aplicación macrofinanciera.

13.2. Resultado esperado

Al finalizar este hito deberá existir una aplicación:

app/streamlit_app.py

capaz de mostrar:

- Resumen macrofinanciero.
- Evolución del tipo de cambio.
- Indicadores bancarios.
- Dolarización.
- Concentración bancaria.

13.3. Arquitectura del hito

El flujo será:

```
DuckDB
|
V
mart
|
V
Streamlit
|
V
Usuario
```

13.4. Paso 1: Instalar Streamlit

Con el entorno virtual activo, ejecutar:

```
pip install streamlit
pip install plotly
```

Actualizar dependencias:

```
pip freeze > requirements.txt
```

13.5. Paso 2: Crear carpeta de aplicación

Verificar que existe:

```
app/
```

Crear el archivo:

```
app/streamlit_app.py
```

13.6. Paso 3: Importar librerías

Agregar:

```
import duckdb
import pandas as pd
import streamlit as st
import plotly.express as px
```

13.7. Paso 4: Definir ruta de DuckDB

Agregar:

```
DB_PATH = "data/warehouse/macrofinanzas.duckdb"
```

13.8. Paso 5: Crear función de consulta

Agregar:

```
def run_query(query):  
  
    '''  
    conn = duckdb.connect(DB_PATH)  
  
    df = conn.execute(  
        query  
    ).fetchdf()  
  
    conn.close()  
  
    return df  
    '''
```

Esta función permite consultar DuckDB desde Streamlit y devolver los resultados como un DataFrame de Pandas.

13.9. Paso 6: Configurar página

Agregar:

```
st.set_page_config(  
    page_title="Dashboard_Macrofinanciero_RD",  
    page_icon="📊",  
    layout="wide"  
)  
  
st.title(  
    "Dashboard_Macrofinanciero_RD"  
)  
  
st.caption(  
    "Proyecto_de_Data_Warehouse_macrofinanciero_con_DuckDB_y_Streamlit"  
)
```

13.10. Paso 7: Cargar datos desde mart

Agregar:

```
resumen = run_query(
    """
    SELECT_*
    FROM_mart.vw_resumen_macrofinanciero
    ORDER_BY_anio_mes
    """
)

fx = run_query(
    """
    SELECT_*
    FROM_mart.vw_tipo_cambio_diario
    ORDER_BY_fecha
    """
)

banca = run_query(
    """
    SELECT_*
    FROM_mart.vw_indicadores_banca
    ORDER_BY_fecha
    """
)

hhi = run_query(
    """
    SELECT_*
    FROM_mart.vw_hhi_bancario
    ORDER_BY_anio_mes
    """
)
```

13.11. Paso 8: Crear KPIs principales

Agregar:

```
ultimo = resumen.sort_values(
    "anio_mes"
).tail(1)

if not ultimo.empty:

    '''
    tasa_promedio = ultimo[
        "tasa_promedio_mes"
    ].iloc[0]

    activos = ultimo[
        "activos_totales_sistema"
    ].iloc[0]

    depositos = ultimo[
        "depositos_sistema"
    ].iloc[0]

    dolarizacion = ultimo[
        "dolarizacion_depositos_sistema"
    ].iloc[0]

    col1, col2, col3, col4 = st.columns(4)

    col1.metric(
        "Tipo_de_cambio_promedio",
        round(tasa_promedio, 2)
    )

    col2.metric(
        "Activos_del_sistema",
        f"{activos:,.0f}"
    )

    col3.metric(
        "Depositos_del_sistema",
        f"{depositos:,.0f}"
    )
```

```
col4.metric(
    "Dolarizaci n _dep sitos",
    f"{dolarizacion:.2%}"
)
'''
```

13.12. Paso 9: Gráfico de tipo de cambio

Agregar:

```
st.subheader(
    "Evoluci n _del _tipo _de _cambio"
)

fig_fx = px.line(
    fx,
    x="fecha",
    y="tasa_promedio",
    color="moneda_cod",
    title="Tipo _de _cambio _promedio _diario"
)

st.plotly_chart(
    fig_fx,
    use_container_width=True
)
```

13.13. Paso 10: Gráfico de indicadores bancarios

Agregar:

```
st.subheader(
    "Indicadores _bancarios"
)

entidades = banca[
    "entidad_nombre"
].dropna().unique()
```

```
entidad_seleccionada = st.selectbox(
    "Seleccione una entidad financiera",
    entidades
)
```

```
banca_filtrada = banca[
    banca["entidad_nombre"]
    ==
    entidad_seleccionada
]
```

```
fig_roa = px.line(
    banca_filtrada,
    x="fecha",
    y="roa",
    title="ROA"
)
```

```
st.plotly_chart(
    fig_roa,
    use_container_width=True
)
```

```
fig_roe = px.line(
    banca_filtrada,
    x="fecha",
    y="roe",
    title="ROE"
)
```

```
st.plotly_chart(
    fig_roe,
    use_container_width=True
)
```

13.14. Paso 11: Gráfico de dolarización

Agregar:

```
st.subheader(
```



```

"Dolarizaci n_bancaria"
)

fig_dolarizacion = px.line(
    banca_filtrada,
    x="fecha",
    y=[
        "dolarizacion_depositos",
        "dolarizacion_cartera"
    ],
    title="Dolarizaci n_de_dep sitos_y_cartera"
)

st.plotly_chart(
    fig_dolarizacion,
    use_container_width=True
)

```

13.15. Paso 12: Gráfico de concentración bancaria

Agregar:

```

st.subheader(
    "Concentraci n_bancaria"
)

fig_hhi = px.line(
    hhi,
    x="anio_mes",
    y="hhi_activos",
    title=" ndice _HHI_por_activos"
)

st.plotly_chart(
    fig_hhi,
    use_container_width=True
)

```

13.16. Paso 13: Mostrar tablas de datos

Agregar:

```
with st.expander(
    "Ver resumen macrofinanciero"
):

    """
    st.dataframe(
        resumen,
        use_container_width=True
    )
    """

with st.expander(
    "Ver indicadores bancarios"
):

    """
    st.dataframe(
        banca,
        use_container_width=True
    )
    """
```

13.17. Paso 14: Versión completa inicial de la aplicación

El archivo `app/streamlit_app.py` debe tener:

```
import duckdb
import pandas as pd
import streamlit as st
import plotly.express as px

DB_PATH = "data/warehouse/macrofinanzas.duckdb"

def run_query(query):
```

```

'''
conn = duckdb.connect(DB_PATH)

df = conn.execute(
    query
).fetchdf()

conn.close()

return df
'''

st.set_page_config(
    page_title="Dashboard_Macrofinanciero_RD",
    page_icon="📊",
    layout="wide"
)

st.title(
    "Dashboard_Macrofinanciero_RD"
)

st.caption(
    "Proyecto_de_Data_Warehouse_macrofinanciero_con_DuckDB_y_Streamlit"
)

resumen = run_query(
    """
SELECT *
FROM mart.vw_resumen_macrofinanciero
ORDER BY anio_mes
    """
)

fx = run_query(
    """
SELECT *
FROM mart.vw_tipo_cambio_diario
ORDER BY fecha
    """
)

```

```

)

banca = run_query(
    """
    SELECT_*
    FROM_mart.vw_indicadores_banca
    ORDER_BY_fecha
    """
)

hhi = run_query(
    """
    SELECT_*
    FROM_mart.vw_hhi_bancario
    ORDER_BY_anio_mes
    """
)

ultimo = resumen.sort_values(
    "anio_mes"
).tail(1)

if not ultimo.empty:

    '''
    tasa_promedio = ultimo[
        "tasa_promedio_mes"
    ].iloc[0]

    activos = ultimo[
        "activos_totales_sistema"
    ].iloc[0]

    depositos = ultimo[
        "depositos_sistema"
    ].iloc[0]

    dolarizacion = ultimo[
        "dolarizacion_depositos_sistema"
    ].iloc[0]

```

```

col1, col2, col3, col4 = st.columns(4)

col1.metric(
    "Tipo de cambio promedio",
    round(tasa_promedio, 2)
)

col2.metric(
    "Activos del sistema",
    f"{activos:,.0f}"
)

col3.metric(
    "Depósitos del sistema",
    f"{depositos:,.0f}"
)

col4.metric(
    "Dolarización depósitos",
    f"{dolarizacion:.2%}"
)

'''

st.subheader(
    "Evolución del tipo de cambio"
)

fig_fx = px.line(
    fx,
    x="fecha",
    y="tasa_promedio",
    color="moneda_cod",
    title="Tipo de cambio promedio diario"
)

st.plotly_chart(
    fig_fx,
    use_container_width=True
)

```

```

st.subheader(
    "Indicadores_bancarios"
)

entidades = banca[
    "entidad_nombre"
].dropna().unique()

entidad_seleccionada = st.selectbox(
    "Seleccione una entidad financiera",
    entidades
)

banca_filtrada = banca[
    banca["entidad_nombre"]
    ==
    entidad_seleccionada
]

fig_roa = px.line(
    banca_filtrada,
    x="fecha",
    y="roa",
    title="ROA"
)

st.plotly_chart(
    fig_roa,
    use_container_width=True
)

fig_roe = px.line(
    banca_filtrada,
    x="fecha",
    y="roe",
    title="ROE"
)

st.plotly_chart(

```

```
fig_roe,
use_container_width=True
)

st.subheader(
"Dolarizaci n_bancaria"
)

fig_dolarizacion = px.line(
banca_filtrada,
x="fecha",
y=[
"dolarizacion_depositos",
"dolarizacion_cartera"
],
title="Dolarizaci n_de_dep sitos_y_cartera"
)

st.plotly_chart(
fig_dolarizacion,
use_container_width=True
)

st.subheader(
"Concentraci n_bancaria"
)

fig_hhi = px.line(
hhi,
x="anio_mes",
y="hhi_activos",
title=" ndice _HHI_por_activos"
)

st.plotly_chart(
fig_hhi,
use_container_width=True
)

with st.expander(
```

```
"Ver_resumen_macrofinanciero"
):

'''
st.dataframe(
    resumen,
    use_container_width=True
)
'''

with st.expander(
    "Ver_indicadores_bancarios"
):

    '''
    st.dataframe(
        banca,
        use_container_width=True
    )
    '''
```

13.18. Paso 15: Ejecutar Streamlit

Desde la raíz del proyecto:

```
streamlit run app/streamlit_app.py
```

Streamlit abrirá una dirección local parecida a:

<http://localhost:8501>

13.19. Paso 16: Verificar funcionamiento

Comprobar:

- El dashboard abre correctamente.
- Los KPIs aparecen en la parte superior.
- El gráfico del tipo de cambio se visualiza.

- El selector de entidad funciona.
- Las tablas aparecen dentro de los expansores.

13.20. Paso 17: Problemas frecuentes

13.20.1. No module named streamlit

Instalar:

```
pip install streamlit
```

13.20.2. No module named plotly

Instalar:

```
pip install plotly
```

13.20.3. La vista mart no existe

Ejecutar primero:

```
python etl/run_pipeline.py
```

13.20.4. La base DuckDB no existe

Verificar que existe:

```
data/warehouse/macrofinanzas.duckdb
```

13.21. Buenas prácticas

Durante este hito seguiremos estas reglas:

1. Streamlit debe consumir la capa mart.
2. No consultar directamente staging.
3. Separar consultas SQL de visualizaciones cuando el proyecto crezca.
4. Mantener el dashboard simple en la primera versión.
5. Validar que el pipeline corre antes de abrir Streamlit.

13.22. Checklist de validación

- ☐ Streamlit instalado.
- ☐ Plotly instalado.
- ☐ Archivo `streamlit_app.py` creado.
- ☐ App conectada a DuckDB.
- ☐ KPIs visibles.
- ☐ Gráficos visibles.
- ☐ Filtro de entidad funcional.
- ☐ Dashboard abierto localmente.

13.23. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Dashboard Streamlit funcional.
- Conexión DuckDB-Streamlit.
- KPIs macrofinancieros.
- Gráficos básicos.
- Primera aplicación analítica del proyecto.

13.24. Próximo hito

En el Hito 11 prepararemos el proyecto para publicación.
Aprenderemos a:

- Organizar archivos para GitHub.
- Crear un README.md profesional.
- Preparar requirements.txt.
- Verificar que la aplicación pueda ejecutarse en otro entorno.
- Preparar el despliegue en Streamlit Community Cloud.

Capítulo 14

Hito 11: Preparación del Proyecto para GitHub y Despliegue

14.1. Objetivo

El objetivo de este hito es preparar el proyecto para ser publicado en GitHub y posteriormente desplegado en Streamlit Community Cloud.

Hasta este punto ya hemos construido:

```
CSV
|
V
Python
|
V
DuckDB
|
V
staging
|
V
core
|
V
mart
```

```
|
V
Streamlit
```

Ahora debemos organizar el repositorio para que cualquier persona pueda entender, instalar y ejecutar el proyecto.

Al finalizar este hito el estudiante será capaz de:

- Revisar la estructura final del repositorio.
- Crear un README.md profesional.
- Actualizar el archivo requirements.txt.
- Preparar el archivo .gitignore.
- Validar que la aplicación se ejecuta desde cero.
- Preparar el proyecto para publicación en GitHub.

14.2. Resultado esperado

Al finalizar este hito el proyecto deberá estar listo para subirse a GitHub.

La estructura esperada será:

```
dwh_macrofinanciero_rd/
```

```
+-- app/
|   -- streamlit_app.py
```

```
+-- data/
|   +-- raw/
|   +-- processed/
|   -- warehouse/
```

```
+-- docs/
|   -- book/
```

```
+-- etl/
|   +-- db.py
|   +-- load_staging.py
```

```

|   +-- load_dimensions.py
|   +-- load_facts.py
|   +-- load_marts.py
|   -- run_pipeline.py

+-- sql/
|   +-- 01_create_schemas.sql
|   +-- 02_staging_ddl.sql
|   +-- 03_dimensions_ddl.sql
|   +-- 04_load_dimensions.sql
|   +-- 05_facts_ddl.sql
|   +-- 06_load_facts.sql
|   -- 07_marts.sql

+-- tests/

+-- README.md
+-- requirements.txt
+-- .gitignore
+-- .env.example

```

14.3. Paso 1: Revisar estructura del proyecto

Desde la raíz del proyecto ejecutar:

```
tree /f
```

En caso de no tener disponible el comando `tree`, usar:

```
dir
```

Verificar que las carpetas principales existen:

- app
- data
- docs
- etl
- sql
- tests

14.4. Paso 2: Revisar el archivo .gitignore

El archivo .gitignore evita subir al repositorio archivos innecesarios, privados o pesados.

Debe contener:

```
.venv/  
.env  
  
**pycache**/  
*.pyc  
*.pyo  
  
.vscode/  
  
data/processed/  
data/warehouse/*.duckdb  
data/warehouse/*.wal  
  
.DS_Store  
Thumbs.db
```

14.5. Nota importante sobre DuckDB

El archivo:

```
data/warehouse/macrofinanzas.duckdb
```

no debe subirse a GitHub si el proyecto crecerá en tamaño.

En su lugar, el usuario debe poder reconstruir la base ejecutando:

```
python etl/run_pipeline.py
```

Esta es una práctica recomendable porque permite reproducibilidad.

14.6. Paso 3: Revisar .env.example

El archivo:

```
.env.example
```

debe documentar la ruta esperada de la base DuckDB.

Contenido sugerido:

```
DUCKDB_PATH=data/warehouse/macrofinanzas.duckdb
```

El archivo real:

```
.env
```

no debe subirse a GitHub.

14.7. Paso 4: Actualizar requirements.txt

Con el entorno virtual activo:

```
pip freeze > requirements.txt
```

Verificar que incluya, al menos:

- duckdb
- pandas
- python-dotenv
- streamlit
- plotly

14.8. Paso 5: Crear un README profesional

Actualizar el archivo:

```
README.md
```

con el siguiente contenido sugerido:

```
# Data Warehouse Macrofinanciero RD
```

```
Proyecto aplicado de ingeniería de datos para construir una plataforma
```

```
## Objetivo
```

Construir un flujo completo de datos que permita:

- * Ingerir datos desde archivos CSV.
- * Almacenarlos en DuckDB.
- * Organizar la información en capas staging, core y mart.
- * Construir indicadores macrofinancieros.
- * Publicar un dashboard interactivo en Streamlit.

Arquitectura

CSV -> Python -> DuckDB -> Staging -> Core -> Mart -> Streamlit

Tecnologías

- * Python
- * DuckDB
- * Pandas
- * Streamlit
- * Plotly
- * Git
- * GitHub

Estructura del proyecto

```
app/  
data/  
docs/  
etl/  
sql/  
tests/
```

Cómo ejecutar el proyecto

1. Crear entorno virtual

```
python -m venv .venv
```

2. Activar entorno virtual

```
.venv\Scripts\activate
```


3. Instalar dependencias

```
pip install -r requirements.txt
```

4. Ejecutar pipeline

```
python etl/run_pipeline.py
```

5. Abrir dashboard

```
streamlit run app/streamlit_app.py
```

Autor

Francisco Ramirez

14.9. Paso 6: Validar ejecución completa

Desde la raíz del proyecto ejecutar:

```
python etl/run_pipeline.py
```

Verificar que se ejecutan correctamente:

- `loadstagging.py`
- `loadddimensions.py`
- `loadfacts.py`
- `loadmarts.py`

14.10. Paso 7: Validar dashboard local

Ejecutar:

```
streamlit run app/streamlit_app.py
```

Verificar:

- La aplicación abre en el navegador.
- Los KPIs se muestran correctamente.
- Los gráficos se renderizan.
- Las tablas se visualizan.
- No aparecen errores de conexión con DuckDB.

14.11. Paso 8: Inicializar Git

Si el repositorio aún no está inicializado:

```
git init
```

Verificar estado:

```
git status
```

14.12. Paso 9: Agregar archivos al repositorio

Ejecutar:

```
git add .
```

Revisar qué archivos serán incluidos:

```
git status
```

Confirmar que no aparecen:

- .env
- .venv/
- macrofinanzas.duckdb

14.13. Paso 10: Crear commit

Ejecutar:

```
git commit -m "Prepare project for GitHub deployment"
```

Verificar historial:

```
git log --oneline
```

14.14. Paso 11: Crear repositorio en GitHub

Entrar a GitHub y crear un repositorio nuevo.

Nombre sugerido:

dwh_macrofinanciero_rd

Seleccionar:

- Repositorio público o privado.
- No inicializar con README si ya existe uno local.
- No agregar .gitignore desde GitHub si ya existe localmente.

14.15. Paso 12: Conectar repositorio local con GitHub

GitHub mostrará instrucciones similares a:

```
git remote add origin https://github.com/usuario/dwh_macrofinanciero_rd.  
git branch -M main  
git push -u origin main
```

Ejecutarlas desde la raíz del proyecto.

14.16. Paso 13: Verificar publicación

Entrar al repositorio en GitHub y verificar que aparecen:

- README.md
- requirements.txt
- carpeta app
- carpeta etl
- carpeta sql
- carpeta docs

14.17. Paso 14: Preparar archivo para despliegue

Streamlit Community Cloud utilizará:

`requirements.txt`

para instalar dependencias y:

`app/streamlit_app.py`

como archivo principal de la aplicación.

Verificar que en GitHub existan ambos archivos.

14.18. Paso 15: Consideración importante sobre datos

Si el archivo `macrofinanzas.duckdb` no se sube a GitHub, entonces la app desplegada no tendrá base de datos a menos que:

- La app construya la base al iniciar.
- El pipeline se ejecute antes del despliegue.
- Se suba una versión pequeña de la base DuckDB.

Para esta primera versión educativa, hay dos opciones.

14.18.1. Opción A: Subir una base pequeña

Permite que la app funcione inmediatamente en Streamlit Cloud.

En este caso se puede permitir temporalmente:

`data/warehouse/macrofinanzas.duckdb`

en GitHub, siempre que sea pequeño y no contenga información sensible.

14.18.2. Opción B: Reconstruir la base

Modificar la app para que, si no encuentra la base, ejecute el pipeline automáticamente.

Esta opción es más robusta, pero requiere más cuidado.

14.19. Recomendación para este proyecto

Para la primera publicación, se recomienda la Opción A:

Subir una base DuckDB pequeña, construida con datos simulados.

Más adelante se puede migrar a una arquitectura más robusta donde la base se reconstruya automáticamente.

14.20. Buenas prácticas

Durante este hito seguiremos estas reglas:

1. No subir credenciales al repositorio.
2. Mantener requirements.txt actualizado.
3. Documentar claramente cómo ejecutar el proyecto.
4. Asegurar que el pipeline reconstruye la base.
5. Usar datos simulados para despliegues públicos.

14.21. Checklist de validación

- ☐ Archivo README.md actualizado.
- ☐ Archivo requirements.txt actualizado.
- ☐ Archivo .gitignore revisado.
- ☐ Pipeline ejecuta correctamente.
- ☐ Dashboard abre localmente.
- ☐ Repositorio Git inicializado.
- ☐ Primer commit de despliegue creado.
- ☐ Repositorio publicado en GitHub.

14.22. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Repositorio GitHub profesional.
- Proyecto documentado.
- Dashboard ejecutable localmente.
- Pipeline reproducible.
- Proyecto listo para desplegar en Streamlit Cloud.

14.23. Próximo hito

En el Hito 12 desplegaremos la aplicación en Streamlit Community Cloud.

Aprenderemos a:

- Conectar GitHub con Streamlit Cloud.
- Configurar el archivo principal de la app.
- Gestionar dependencias.
- Resolver errores comunes de despliegue.
- Obtener una URL pública del dashboard.